

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 1 de Rust

Descubriendo la programación y

Filtros digitales

Recomendado a partir de los 15-16 años años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. 🦀 Introducción	1
1.1. 📝 Descripción de los capítulos	1
1.2. Qué es un repositorio y qué es Git	1
1.2.1. ⬇ Instalación de Git en Linux	2
1.2.2. ⬇ Instalación de Git en MacOS	2
1.2.3. ⬇ Instalación de Git en Windows	3
1.3. ⬇ Descargar el material asociado a los cuadernos con Git	3
1.4. 📁 Estructura del Repositorio (curso-rust-institutos)	3
2. 🦀 Qué es un programa y por qué elegimos Rust	4
3. 🦀 Conceptos que necesitarás recordar	5
3.1. La información que maneja un programa	5
3.1.1. Tipos de datos reales	5
3.2. 🗣 El altavoz de tu código: Salida estándar con println!()	6
3.2.1. Los «Anclajes» u «Ojos de buey» {}	6
3.2.2. El limitador de decimales {:.N}	7
3.3. El proceso de compilación de un programa	7
3.4. Qué es una variable en programación	9
3.4.1. Variables inmutables	10
3.4.2. Variables mutables	10
3.5. Las notas que el ordenador ignora: Los comentarios	11
3.6. Cuando los planes fallan: Errores de escritura vs. Errores de lógica	11
3.7. Divide y vencerás: ¿Qué es una función (fn)?	12
4. 🦀 A los mandos de la Playground	13
4.1. El panel de control	13
4.2. El botón mágico: [SHARE]	14
4.3. Activando superpoderes: ¿Qué es un crate?	14
4.4. Lanzar el dado sin utilizar crates	16
4.5. Aprendiendo a leer el «idioma» del compilador	16
4.6. Tu copiloto digital: Cómo preguntar a la IA	17
4.7. 🛑 Las 3 reglas para no volverte dependiente de la IA	18
5. 🦀 Laboratorio	19
5.1. Sección 1: Despegue	19
5.1.1. El programa hola mundo	19
5.1.2. Qué hacer cuando obtenemos errores del compilador	21
5.1.3. Tipos e inferencia	22
5.1.4. Mis primeras funciones	24
5.2. Sección 2: Cajas y permisos	27
5.2.1. La palabra <code>let</code> para variables inmutables	27
5.2.2. Las palabras <code>let mut</code> para variables mutables	28
5.2.3. Prácticas sobre permisos en el laboratorio	29

5.3.	Sección 3: Tomar decisiones en el código (if, else if, else)	30
5.3.1.	Tomar decisiones en base a preguntas en el código	30
5.4.	Sección 4: El bucle for con rangos	31
5.5.	Sección 5: Bucles infinitos loop y while: repetir tareas sin cansarse	33
5.5.1.	El bucle loop	33
5.5.2.	El Bucle while	34
5.6.	Sección 5: Proyectos clave	36
5.6.1.	Conversor de unidades interactivo	36
5.6.2.	Geneador de historias aleatorias (tipo Mad Libs)	37
6.	 Tipos de datos y operadores en Rust	39
6.1.	Tipos de datos en Rust	39
6.1.1.	 ¿Qué es un bit?	39
6.1.2.	 La memoria del ordenador y el sistema binario	39
6.1.3.	 Tipos de datos por defecto	40
6.1.4.	 Las dos familias de tipos en Rust	41
6.1.4.1.	Tipos Escalares (Un solo valor)	41
6.1.4.2.	Tipos Compuestos (Varios valores juntos)	41
6.1.5.	Ejemplo que utiliza diferentes tipos	42
6.1.6.	Gestión de Texto: La Diferencia entre &str y String	44
6.1.6.1.	El tipo &str (Referencia de cadena)	44
6.1.6.2.	2. El tipo String	44
6.2.	Los motores del cambio: Operadores en Rust	45
6.2.1.	 Operadores Matemáticos (Los de toda la vida)	45
6.2.2.	 Operadores de Comparación (Los jueces)	45
6.2.3.	 Operadores Lógicos (Los detectives)	46
6.2.4.	 Ejercicio propuesto: Misión «Acceso a la Mazmorra del Dragón»	46
7.	 Subiendo de nivel: Instalación de un Entorno de Desarrollo Profesional	48
7.1.	 Instalación en LINUX	48
7.2.	 Instalación en WINDOWS	49
7.3.	 Instrucciones en macOS	50
7.4.	 Configuración de Visual Studio Code (Para todos los sistemas)	50
7.5.	 La Gran Comprobación: ¿Funciona todo?	52
8.	 Tratamiento de imágenes con Rust	53
8.1.	Creando tu primer proyecto con Cargo	53
8.2.	 El cuadro de mandos de Cargo	54
8.3.	¿  Qué es una imagen para un ordenador?	54
8.4.	Programa 1: Leyendo el color de un píxel	55
8.5.	Programa 2: Modificando píxeles y pintando un cuadrado rojo	57
8.6.	Programa 3: Recorriendo la matriz completa (El filtro Negativo)	59
9.	Conceptos avanzados en Rust	63
9.1.	Ámbito (Scope) y la Destrucción de Datos (drop)	65
9.2.	Conflicto: Préstamo Mutable e Intento de Lectura Posterior	66
9.3.	Pasar Variables a Funciones (&, &mut y Move)	68

9.4. El Comportamiento en los Bucles for (Iteradores)	69
9.5. Iteradores que devuelven referencias (Ejemplo Extra)	70
9.6. El struct Persona y los «superpoderes» automáticos	71
9.7. ¿Qué son los traits?	73

1. Introducción

¡Te damos la bienvenida al fascinante mundo de la programación!

Tienes en tus manos el primer cuaderno de una serie que iremos publicando paso a paso para acompañarte en esta aventura. No necesitas ninguna experiencia previa, ni saber de informática, ni ser un genio de las matemáticas. El único objetivo de este cuaderno es romper el miedo. Vamos a descubrir juntos cómo piensa un programador y cómo dar tus primeras órdenes a un ordenador.

Para que no tengas que complicarte al comenzar instalando programas raros ni configurando pantallas difíciles, trabajaremos directamente desde una página web en Internet. Usaremos la aplicación web **Rust Playground** como si fuera un lienzo en blanco para pintar nuestras ideas, crear historias y diseñar nuestros propios juegos. ¡Empezamos!

1.1. Descripción de los capítulos

Para que no te pierdas en esta aventura, hemos dividido este cuaderno en varias etapas diseñadas para ir ganando «superpoderes» como programador poco a poco:

- **El Despegue (Capítulos 1 al 4):** Aprenderemos cómo se comunica un programador con el ordenador, dónde guardamos la información y cómo usar la Playground de Rust desde el navegador sin instalar nada.
- **El Laboratorio de Pruebas (Capítulo 5):** Nos remangaremos para experimentar con código real, aprenderemos a tomar decisiones con condiciones y a repetir tareas usando bucles en proyectos divertidos.
- **Profundizando en Rust (Capítulo 6):** Viajaremos al interior de la memoria del ordenador para entender los tipos de datos y los operadores matemáticos y lógicos que hacen que la magia funcione.
- **El Entorno Profesional (Capítulo 7):** Dejaremos atrás el navegador de Internet para instalar **Rust** y **Visual Studio Code** en tu ordenador, como tienen en realidad los ingenieros de software.
- **Tratamiento de imágenes (Capítulo 8):** ¡Crearemos programas capaces de leer y aplicar filtros visuales a fotos reales!
- **Conceptos avanzados en Rust (Capítulo 9):** Analizaremos el concepto fundamental de **préstamo** y **propiedad** en Rust, y haremos además una introducción a los **struct** y los **trait**.

1.2. Qué es un repositorio y qué es Git

Cuando los programadores trabajan en un proyecto (como un videojuego o este curso de Rust), escriben decenas de archivos con código y/o texto. Para que no se vuelva un caos de carpetas y archivos, con nombres como: proyecto_final_version3_definitiva_ESTA_SI.rs utilizamos una herramienta mágica llamada **Git**.

- **¿Qué es Git?:** Git es como una máquina del tiempo para tus archivos de código. Se encarga de vigilar tu proyecto y guardar «puntos de control» (como en los videojuegos). Si

haces un cambio en tu código o el programa se rompe y no sabes qué ha pasado, puedes pulsar un botón y volver exactamente al momento en que todo funcionaba perfectamente. Sin embargo, nosotros no utilizaremos Git para realizar esta tarea, que se denomina **control de versiones**. Lo utilizaremos únicamente para descargarnos todo el material asociado al **Cuaderno 1** desde Internet, hasta una carpeta que elijamos en nuestro ordenador personal.

Ejercicio:

 Crea una carpeta denominada `proyectos-rust` en cualquier parte de tu disco duro, en donde almacenarás todos los archivos que vayamos escribiendo.

- **¿Qué es un Repositorio?:** Es el lugar donde se guardan todos los archivos de un proyecto junto con todo el historial de su «máquina del tiempo». Es decir, el repositorio guarda para cada archivo, todas las versiones por las que ha pasado a través del tiempo hasta su estado actual.
- **¿Qué es GitHub?:** **Git** funciona en tu propio ordenador, pero si quieres compartir tu caja fuerte digital de ficheros con el mundo, necesitas una nube. **GitHub** es como el «Instagram» o la red social de los programadores. Es una plataforma en Internet donde guardamos nuestros repositorios para que otras personas puedan ver nuestro código, descargarlo o colaborar con nosotros. Todo el material de este curso está guardado en un repositorio de GitHub llamado **curso-rust-institutos**. Luego explicaremos cómo puedes descargarlo a tu ordenador.

1.2.1. Instalación de Git en Linux

Si usas Linux, instalar Git es increíblemente rápido desde la terminal. Abre tu terminal favorita y escribe el comando según tu sistema:

- En **Ubuntu / Debian / Linux Mint**:

```
sudo apt update && sudo apt install git -y
```

- En **Fedora**:

```
sudo dnf install git -y
```

Para comprobar que se ha instalado bien, escribe el comando `git --version`. Si sale un número de versión, ¡ya lo tienes!

1.2.2. Instalación de Git en MacOS

En Mac, el propio sistema te lo pone muy fácil.

1. Abre la Terminal (búscala en el Launchpad o con Spotlight presionando `Cmd + Espacio`).
2. Escribe el siguiente comando y pulsa Enter:

```
git --version
```

3. Si no lo tienes instalado, aparecerá una ventana emergente de Apple preguntándote si deseas instalar las **Herramientas de línea de comandos de Xcode**. Haz clic en Instalar y acepta las condiciones. ¡El sistema hará todo el trabajo por ti!

Para comprobar que se ha instalado bien, escribe el comando `git --version`. Si sale un número de versión, ¡ya lo tienes!

1.2.3. Instalación de Git en Windows

Para instalar Git en Windows, seguiremos el método oficial:

1. Entra en la página web oficial: git-scm.com y descarga el instalador para Windows.
2. Ejecuta el archivo descargado. Te saldrá un asistente con muchas opciones de configuración. No te compliques: haz clic en «Next» (Siguiente) a todo dejando las opciones por defecto hasta que termine la instalación.
3. Al finalizar, tendrás un nuevo programa instalado llamado **Git Bash**. Es una terminal especial para Windows desde la cual podrás usar todos los comandos de Git exactamente igual que si estuvieras en Linux o Mac.

Existen varias formas de abrir la terminal **Git Bash** en Windows pero unas de las más sencillas son las siguientes:

- **Desde el menú Inicio:** Haz clic en el botón de Inicio de Windows, busca la carpeta **Git** en la lista de aplicaciones y haz clic sobre el icono de **Git Bash**.
- **Desde el Explorador de archivos (En cualquier carpeta):** Haz clic derecho en una zona vacía de cualquier carpeta o en el Escritorio. En el menú contextual, selecciona **Git Bash Here** (O Mostrar más opciones > **Git Bash Here** en Windows 11). Esto abrirá la terminal directamente ubicada en esa ruta.

Para comprobar que **Git** se ha instalado bien, escribe el comando `git --version`. Si sale un número de versión, ¡ya lo tienes!

1.3. Descargar el material asociado a los cuadernos con Git

Ahora que ya tienes la herramienta Git en tu ordenador, vas a utilizar tu primer comando de programación para traerte una copia exacta de toda nuestra caja fuerte del curso a tu equipo. A esto los programadores lo llamamos clonar un repositorio.

Abre tu terminal —en la carpeta `proyectos-rust` que hemos creado antes o en cualquier otra— (o Git Bash si estás en Windows) y escribe la siguiente orden mágica:

```
git clone https://github.com/vsorianos58/curso-rust-institutos.git
```

En la carpeta desde la que has lanzado el comando **git clone** —debería ser: **proyectos-rust**—, debe aparecerte una nueva carpeta con el nombre **curso-rust-institutos**. Contiene el material asociado a los Cuadernos de Rust. Este material se comparte a la vez que se actualiza por lo que si lo descargas en sucesivas ocasiones su contenido será distinto hasta que tengamos una versión definitiva.

1.4. Estructura del Repositorio (curso-rust-institutos)

(Esto lo completaremos al final)

2. Qué es un programa y por qué elegimos Rust

Un programa informático es, en realidad, algo muy parecido a una receta de cocina. Es una lista ordenada de instrucciones detalladas que le damos a un ordenador para que realice una tarea concreta. Tu videojuego favorito, la aplicación de TikTok que usas en el móvil, el buscador de Google o el sistema de frenos de un coche moderno... todo eso son programas. Los ordenadores son máquinas increíblemente rápidas y potentes, pero tienen un pequeño defecto: no poseen imaginación ni iniciativa propia. Un ordenador no hace lo que tú quieres que haga; hace exactamente lo que tú le ordenas que haga en el programa. Si le das una orden equivocada obtendrás una respuesta que seguramente no esperabas.

Para escribir estas recetas de instrucciones existen cientos de «lenguajes» diferentes que llamamos lenguajes de programación. En este curso hemos elegido uno llamado **Rust**, y lo hemos hecho por tres razones muy potentes:

- **Es un lenguaje del futuro (y del presente):** Las empresas tecnológicas más importantes del mundo (como Microsoft, Google o la NASA) están usándolo ahora mismo para construir sus sistemas más modernos y seguros. Aprender Rust hoy es aprender la tecnología que dominará los próximos 20 años.
- **Es súper seguro:** A diferencia de otros lenguajes antiguos, Rust tiene un «asistente integrado» (el compilador) que es extremadamente estricto. Si cometes un error al escribir, Rust te avisará amablemente antes de que el programa se rompa, explicándote qué has hecho mal y cómo solucionarlo. Es como tener un tutor personal a tu lado.
- **Es ultra rápido:** Con Rust se pueden crear programas que se ejecutan a la velocidad del rayo consumiendo poquísima memoria, lo que lo hace ideal desde para programar naves espaciales hasta para los videojuegos con los gráficos más exigentes.

A continuación, vamos a ver cómo se organiza la información que va a manejar nuestro primer programa Rust.

3. Conceptos que necesitarás recordar

En este apartado vamos a incluir algunos conceptos sobre Rust de la máxima importancia. Evidentemente no están todos los que son pero si un pequeño conjunto que te resultará crucial para entender el resto del cuaderno y podrás volver aquí para revisarlos cada vez que lo necesites.

3.1. La información que maneja un programa

Los programas informáticos se caracterizan por manejar información. Esta información pueden obtenerla de muchas formas, por ejemplo: puede estar incrustada en el mismo programa o, facilitada por el usuario a través del ratón o el teclado, mediante una conexión a una cámara web, mediante la lectura de un código de barras, mediante la lectura de ficheros que recibe desde Internet o desde otras vías; existen muchas otras formas mediante las cuales un programa puede recibir información.

En base a las fuentes de información citadas anteriormente, podríamos clasificar la información que puede recibir —y por tanto manejar— un programa en muchas categorías pero, vamos a simplificar este proceso y a distinguir solamente tres categorías de información:

- Numérica (representada por números enteros y decimales)
- Texto (representado por una agrupación de letras o caracteres)
- Booleanos, que solo tienen dos valores (verdadero y falso)

Lo primero a resaltar es que mientras en matemáticas el conjunto de los números es infinito, en informática no lo es y tiene valores máximos y mínimos que pueden alcanzar.

Un texto puede contener numerosos caracteres. Considérense los caracteres que contiene un libro cuyo contenido puede ser manejado íntegramente por un programa informático.

Sin embargo, el conjunto de los booleanos solo tiene dos valores, **true** (verdadero) y **false** (falso) pero, en informática son tan valiosos como los anteriores.

3.1.1. Tipos de datos reales

En el apartado anterior hablamos de forma muy general de **números**, **textos** y **booleanos**. Sin embargo, para programar de verdad en Rust, debemos entender que el ordenador solo entiende de interruptores eléctricos (**unos** y **ceros**).

 **Nota:** Más adelante en el tema, en el apartado 6.1.- Tipos de datos en Rust, desarrollaremos con mayor rigor el concepto que introducimos en este apartado. Lo hacemos así para que puedas entender los ejemplos de programas que vamos a estudiar en el siguiente tema sin adentrarnos todavía en una justificación teórica que es algo compleja y que dejamos para más tarde, cuando ya hayas utilizado estos conceptos desde la práctica con los ejemplos.

Para que esos unos y ceros se conviertan en una **edad**, en tu **nombre** o en el **precio** de un videojuego, Rust necesita saber el **Tipo de Dato**. Un tipo de dato es, simplemente, la plantilla

que le dice al ordenador cómo traducir esos unos y ceros de la memoria RAM para poder entenderlos.

Aunque existen muchos tipos, por ahora solo necesitas dominar de momento estos cuatro tipos sagrados:

- **Enteros (i32):** Sirven para contar cosas completas que no se pueden partir. Tu edad, el año actual o el número de vidas de un personaje. Rust los llama **i32** (un entero de 32 bits). En la memoria ocupan por tanto un espacio de tamaño fijo.
- **Decimales (f64):** Sirven para medir cosas precisas. El precio de una golosina (1.50), la distancia en millas (5.34) o la temperatura (23.6). Rust los llama **f64** (número en coma flotante de 64 bits). El ordenador los guarda usando una parte para el número y otra para saber dónde va el punto decimal. ¡Ojo! En programación usamos el punto ., nunca la coma , para los decimales.
- **Texto (&str y String):** Son cadenas de letras, números y símbolos unidos, siempre encerrados entre comillas (como "Halcón68"). En la memoria, el ordenador los guarda como una lista de caracteres seguidos.
- **Booleanos (bool):** Es el tipo más sencillo del mundo. Solo tiene dos valores posibles: **true** (verdadero) o **false** (falso). En la memoria ocupan el mínimo espacio posible: un único interruptor encendido o apagado. Sirven para tomar decisiones (¿el juego ha terminado?, ¿el usuario es mayor de edad?).

3.2. 📢 El altavoz de tu código: Salida estándar con println!()

Un programa que calcula cosas en silencio dentro del ordenador pero no te las muestra es un programa inútil. Necesitamos un **altavoz** para comunicarnos con el usuario humano, y en Rust ese altavoz es la orden **println!()**.

La palabra viene del inglés Print Line (imprimir línea). Su funcionamiento parece sencillo, pero esconde dos herramientas de diseño avanzadas que verás constantemente en el laboratorio:

3.2.1. Los «Anclajes» u «Ojos de buey» {}

Cuando quieres mostrar texto combinado con variables, no puedes tirarlo todo junto. Rust utiliza las llaves **{}** como marcadores de posición o **huecos para rellenar**. Piensa en ellos como un gancho donde vas a colgar tus datos en orden. Por ejemplo:

```
println!("Hola {}, tienes {} años.", nombre, edad);
```

Rust leerá esa línea y la imprimirá en la pantalla colocando el valor de `nombre` en el primer `{}` y el valor de `edad` en el segundo `{}`. Si cambias el orden de las variables al final, la frase cambiará por completo.

Por ejemplo, si `nombre` vale "Juan" y `edad` vale 25, la orden anterior imprimiría por pantalla:

```
Hola Juan, tienes 25 años.
```

3.2.2. El limitador de decimales {:.N}

A los ordenadores se les da tan bien la precisión matemática que, si calculas una división o una conversión de kilómetros a millas, a veces te arrojan resultados como 8.59432210943. ¡Eso en una pantalla queda feísimo!

Para solucionarlo, Rust te permite tunear el marcador {} añadiendo instrucciones dentro. Una de las más utilizadas es {:.2}.

- Los dos puntos : en el interior significan: «Atención, voy a darte una instrucción de formato».
- El punto y el número dos .2 significan: «Corta el número decimal y muestra solo dos dígitos después del punto».
- Se puede generalizar a N decimales {:.N}, donde N puede valer 2, 3, 4, etc.

Ejemplo:

Si **resultado** vale 5.14367, la siguiente línea,

```
println!("Distancia: {:.3} km", resultado);
```

mostraría:

Distancia: 5.143 km

3.3. El proceso de compilación de un programa

Un poco más adelante te explicaremos de forma más detallada el funcionamiento de la **Playground de Rust**. Es básicamente una aplicación web que te permite escribir en el lado izquierdo tu programa Rust y, si no tienes errores, cuando le das al botón **[RUN]** ves en la parte derecha el resultado de ejecutar el programa.

Si cometes errores en la escritura de tu programa la Playground no generará ningún resultado. Te marcará los mensajes de error en rojo y te dará normalmente pistas para subsanarlos. Es posible que no sigas las recomendaciones de lo que Rust considera un estilo perfecto. En tal caso te entregará mensajes en color amarillo para avisarte pero en este caso si que ejecutará el programa entregándote los resultados.

Vamos a utilizar por primera vez la Playground.

1. Pulsa sobre el enlace: [Rust Playground oficial](#)

En general, el programa que nosotros escribimos se denomina programa fuente. El primero será simplemente imprimir un saludo por la pantalla.

2. Copia el texto que tienes abajo y pégalo en la parte izquierda de la ventana de la **Playground**, borrando primero el contenido previo en caso de existir.

```
fn main() {
    println!("¡Bienvenido al curso de Rust!");
}
```

❗ **Nota:** para borrar en la Playground, sigue los dos pasos que vienen a continuación

- Click con el botón derecho del ratón sobre el panel izquierdo de la pantalla y elige **Seleccionar todo** en el menú contextual.
 - Pulsa la Tecla Supr
3. Pulsa con el ratón sobre el botón **[RUN]**

Verás aparecer en la parte derecha el siguiente texto:

```
Standard Error
  Compiling playground v0.0.1 (/playground)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.52s
  Running `target/debug/playground`
Standard Output
¡Bienvenido al curso de Rust!
```

El resultado de tu programa es solamente la última línea:

```
¡Bienvenido al curso de Rust!
```

4. Explicación

Si estuviéramos trabajando en un entorno de programación más complejo que la **Playground**, seguramente el código que hemos copiado y pegado se encontraría en un fichero llamado `main.rs`. **rs** es la extensión que utilizan los ficheros de Rust. Sin embargo, al trabajar con la Playground se simplifica este proceso y desaparece el concepto de fichero `main.rs`.

Nosotros trabajaremos sencillamente con el concepto de programa, identificado con el texto que escribimos en la parte izquierda de la Playground.

Dentro de nuestro programa, en la instrucción **`fn main()`**, la palabra **`fn`** sirve para definir una función (ya veremos qué es esto) y la instrucción **`println!("¡Bienvenido al curso de Rust!");`** imprime el mensaje **`¡Bienvenido al curso de Rust!`** (sin las comillas) por la pantalla.

Este **programa** fuente está escrito en un lenguaje **Rust** que el ser humano puede entender. Sin embargo, el ordenador solo entiende de unos y ceros. Para que el ordenador pueda entender y procesar el programa, existe un proceso denominado compilación que convierte el programa fuente en otro equivalente que solo contiene unos y ceros y que el ordenador es capaz de entender.

Podemos llamar a este fichero **ejecutable** aunque no sea totalmente exacto en todos los casos.

Podríamos resumirlo en una frase diciendo que «compilar un programa fuente lo convierte en un programa ejecutable». Esta traducción la realiza un programa denominado **compilador** y es el que genera los mensajes de error cuando nos equivocamos al escribir el programa fuente. El programa ejecutable, como su nombre indica, puede ser ejecutado por nuestro sistema operativo, el sistema bajo el cual se ha compilado, de una forma prácticamente idéntica en Linux, MacOS o Windows.

El programa fuente, el que entiende el humano, es idéntico en las tres plataformas anteriores pero al compilar, cada plataforma genera un código ejecutable que solo ella es capaz de entender.

Como nosotros utilizaremos la **Rust Playground** desde la Web, da igual el sistema operativo desde el que accedamos a esta aplicación web: no se van a generar ficheros compilados que nosotros podamos ver y todos estaremos en las mismas condiciones. Por supuesto, podremos intercambiar entre nosotros los programas fuente como el que acabamos de ver, independientemente de la plataforma desde la que los hayamos creado y en la que vayamos a ejecutarlos.

3.4. Qué es una variable en programación

Reconocemos que es un poco aventurado explicar qué es una variable en programación sin apenas haber escrito código. Sin embargo, partimos de la base de que ya has utilizado variables al resolver problemas en matemáticas o física. Ese concepto, aunque no es exactamente igual en informática, es un punto de partida perfecto.

En clase de física, por ejemplo, decimos:

- $v = 50.5$ (Velocidad en Km/h)
- $t = 2$ (Tiempo en horas)
- e : Espacio recorrido (en Km)

Para calcular el espacio, aplicas la fórmula:

$$e = v * t = 50.5 * 2 = 101Km$$

En física usas esas letras para guardar números. En informática hacemos lo mismo, pero con una gran diferencia: el ordenador necesita guardar esos valores dentro de su memoria RAM y necesita saber qué tipo de información va a meter dentro.

Imagina que la memoria del ordenador es un almacén gigante lleno de **cajas de cristal** para almacenar **variables**. Para usar una caja, necesitas hacer dos cosas:

1. Ponerle una etiqueta con un nombre (para encontrarla rápido)
2. Decirle al ordenador qué tipo de objeto vas a guardar dentro (un número entero, un número con decimales, un texto...), porque cada tipo de dato necesita una caja de tamaño diferente.

Mira cómo escribiríamos el problema de física anterior en Rust.

Copia este código en tu Playground y pulsa **[RUN]**:

```
fn main() {
    let v = 50.5;
    let t = 2.0;
    let e = v * t;
    println!("Espacio recorrido = {} Km", e);
}
```

Con **fn** definimos la función **main()** que es el punto de entrada al programa, por donde empieza a ejecutarse. Lo que hay entre las llaves **{}** es el código de la función **main()**, lo que se ejecutará al pulsar **[RUN]**.

- **¿Qué significa let?** Es la palabra que usamos en **Rust** para **fabricar** una **caja nueva** en la memoria. **let v = 50.5;** significa: «Créame una caja llamada **v** y guarda dentro el número decimal **50.5**».
- **¿Por qué es genial Rust?** Te habrás fijado en que no le hemos dicho a Rust qué tipo de caja queríamos. Rust es un lenguaje inteligentísimo: ve que ponemos un 50.5 (con un punto decimal) y él solo deduce: «¡Ah! Esto es un número decimal, usaré una caja para decimales». Esto se llama inferencia de tipos. No siempre puede proceder así. A veces el programador debe indicar el tipo de dato que va a meter en la caja.
- **¿Qué pasa si nos equivocamos de caja?** Imagina que vas al Traductor de Google, escribes una frase en español pero le dices al programa que está en alemán y que la traduzca al inglés. La traducción será un desastre absoluto. Con el ordenador pasa igual: si intenta leer un texto como si fuera un número, o viceversa, se volverá loco. Por eso Rust es tan estricto con los tipos de datos.

Veremos más adelante que no siempre Rust infiere los tipos de datos a utilizar, e incluso a veces, queremos elegir nosotros que utilice un tipo de datos concreto. En este caso se lo tendremos que indicar de una forma explícita y coherente

3.4.1. Variables inmutables

Por defecto, en Rust, todas las cajas que creas con la palabra **let** son cajas fuertes de cristal. Puedes ver lo que hay dentro, puedes usar su valor para hacer operaciones (como multiplicar **v x t**), pero está completamente prohibido cambiar lo que hay dentro una vez que lo has guardado. Se llaman variables **inmutables**.

Si después de escribir **let t = 2.0;** intentas poner en la línea de abajo **t = 3.0;** para decir que ha pasado una hora más, «el compilador de Rust detendrá el programa, se enfadará y te mostrará un error en letras rojas». Rust hace esto para protegerte: si una variable no debería cambiar, se asegura de que nadie la modifique por error.

3.4.2. Variables mutables

¿Pero qué pasa si estamos programando un videojuego y queremos guardar la puntuación del jugador? La puntuación empieza en 0, pero cambiará cada vez que elimine a un enemigo. «¡Necesitamos una caja que nos permita cambiar su contenido!».

En Rust, para que una caja sea **modificable**, tenemos que añadir la palabra mágica **mut** (de mutable).

Mira este ejemplo:

```
fn main() {
    let mut puntuacion = 0; // Creamos la caja mutable y empieza en 0
    println!("Puntuación inicial: {}", puntuacion);

    puntuacion = 10; // ¡Ahora sí nos deja cambiar el valor de la caja!
    println!("¡Has ganado puntos!: {}", puntuacion);
}
```

❗ **Nota:** Todo lo que hay a la derecha de las dos barras // (incluidas) hasta el final de la línea es un comentario y el programa lo ignora.

Al añadir la palabra **mut**, le estás diciendo a Rust: «Ojo, el contenido de esta caja va a estar cambiando a lo largo del programa, prepárate».

3.5. Las notas que el ordenador ignora: Los comentarios

Cuando escribimos una receta de cocina, a veces añadimos notas al margen como «cuidado, no dejar quemar» o «el horno debe estar muy caliente». En programación hacemos exactamente lo mismo utilizando los **comentarios**.

Un comentario es un fragmento de texto que escribimos dentro de nuestro programa para explicarnos a nosotros mismos (o a un compañero) qué hace el código. Lo maravilloso de los comentarios es que **el compilador los ignora** por completo. Al traducir nuestro programa a unos y ceros, el compilador hace como si esas líneas no existieran.

En Rust, para escribir un comentario de una sola línea solo tenemos que poner dos barras inclinadas //. Todo lo que escribas a la derecha de esas dos barras, incluyendo las dos barras, se volverá de un color diferente en la pantalla y el ordenador no lo ejecutará.

```
fn main() {
    // Esto es un comentario. El ordenador no lo leerá.
    println!("{}", "¡Hola!"); // Aquí imprimimos el saludo.
}
```

También podemos escribir comentarios de varias líneas rodeándolos entre los caracteres /* y */ como se muestra a continuación:

```
fn main() {
    /*Esto es un comentario de varias líneas.
    El compilador no lo leerá y actuará
    como si no estuviera.*/
    println!("{}", "¡Hola!"); // Aquí imprimimos el saludo.
}
```

3.6. Cuando los planes fallan: Errores de escritura vs. Errores de lógica

Para no frustrarte cuando programes, debes saber que existen dos formas muy diferentes de equivocarse en programación. Los programadores se enfrentan a ellas a diario:

1. Errores de Compilación (Fallas de escritura):

Ocurren cuando escribes algo mal en el lenguaje Rust (por ejemplo, olvidar un punto y coma ;, cerrar mal un paréntesis o escribir println en vez de println!). El compilador, que es un árbitro implacable, se dará cuenta inmediatamente, detendrá el proceso y te mostrará letras rojas en la Playground. El programa ni siquiera llega a compilarse. Es el equivalente a intentar leer una frase con palabras inventadas que no existen en el diccionario.

1. Errores de Lógica (El programa «funciona», pero hace lo que quiere):

Estos son los más tramposos. Ocurren cuando tu código está perfectamente escrito (no hay faltas de ortografía en Rust), el compilador lo traduce sin protestar a unos y ceros, le das a RUN... pero el resultado no es el que tú esperabas. Por ejemplo, si querías crear un programa que sumara dos números y por error pusiste el signo de restar -. Para el ordenador el programa es perfecto, pero para tu objetivo es un fracaso. Aquí el error no es de la máquina, ¡es de nuestra lógica!

3.7. Divide y vencerás: ¿Qué es una función (fn)?

Imagina que estás cocinando siguiendo una receta gigante. En lugar de escribir paso a paso cómo se hace una masa de pizza cada vez que la necesitas, en tu libro de cocina tienes una nota que dice: «Hacer la masa de pizza (ver página 20)».

En programación, esa receta independiente se llama **Función**. Una función es un bloque de código aislado que tiene un nombre, hace una tarea muy concreta y se puede reutilizar todas las veces que quieras en tu programa y en otros programas.

Hasta ahora has visto la función mágica **fn main()**. Esa es la función «jefa» de Rust, el punto de partida donde el ordenador empieza a ejecutar el programa. Pero los programadores profesionales nunca meten miles de líneas de código dentro de main. En su lugar, dividen el programa en trocitos pequeños, **funciones**, por tres razones sagradas:

- **Ahorrar trabajo:** Si tienes que calcular el IVA de un producto diez veces, no escribes la fórmula matemática diez veces. Creas la función `calcular_iva` y la llamas cuando la necesitas.
- **Evitar errores:** Si la fórmula del IVA cambia en el futuro, solo tienes que corregirla en un único sitio de tu programa (dentro de su función), no en las diez partes del programa que la utilizas.
- **Hacer el código legible:** Es mucho más fácil leer un programa que dice `abrir_puerta();` y `encender_luces();` que ver cincuenta líneas de programa que realizan estas dos acciones.

4. A los mandos de la Playground

A continuación se describen las distintas partes y funciones de la **Rust Playground** en donde editarás y ejecutarás los programas de **Rust**.

4.1. El panel de control

Imagínate que la Playground de Rust es como el salpicadero de una nave espacial o el menú de configuración de tu videojuego favorito: está diseñado para que tengas todo lo necesario a la vista y al alcance de un clic. Cuando entres en la web, verás que la pantalla está dividida principalmente en dos grandes zonas y una barra de herramientas superior.

Accede ahora a la Playground: [Rust Playground oficial](https://playground.rust-lang.org/)

1. El lienzo de escritura (La parte izquierda): Es tu zona de creación. Es una gran caja blanca (o gris oscura, según el modo que elijas) que funciona como un editor de textos. Todo el código fuente que escribas, modifiques o copies se queda ahí. Verás que las líneas están numeradas a la izquierda (1, 2, 3...); esto es súper útil para saber exactamente en qué línea estás trabajando y para encontrar los errores cuando te los muestre el compilador juunto al número de línea que contiene el error.

2. La pantalla de resultados o Consola (La parte derecha / inferior): Es donde el ordenador te responde. Cuando la Playground termine de procesar tu receta de código, la consola se encenderá de color gris oscuro para mostrarte las respuestas del programa (lo que llamamos la salida estándar o Standard Output) o, si te has equivocado, los mensajes de error con las pistas del compilador.

3. Los botones de mando (La barra superior): En la parte de arriba tienes los botones que activan los **superpoderes** de Rust. Aunque hay varias opciones, para ser un piloto experto solo necesitas dominar estos tres botones clave:

- El botón **[RUN]** (Ejecutar): Es el botón de acción principal. Se visualiza cuando tienes una función `main()` en el panel izquierdo. Al pulsarlo, le ordenas a los servidores de Rust que cojan tu código fuente de la izquierda, lo compilen en milisegundos a unos y ceros y te muestren el resultado de ejecutar inmediatamente el programa*en el panel de la derecha. Truco de programador: Puedes pulsar las teclas **Ctrl + Enter** en tu teclado para ejecutar la acción equivalente a pulsar el botón **[RUN]** sin usar el ratón.
- El botón **[TOOLS] > Rustfmt** (Poner el código bonito): Escribiendo rápido es normal que el código se vea desordenado. Al pulsar sobre **Rustfmt** desplegando el botón **[TOOLS]**, la herramienta utiliza un asistente llamado `Rustfmt` que reordena y limpia visualmente tu código de forma automática. ¡Magia! Tu código se verá limpio, profesional y elegante al instante. Acostúmbrate a pulsarlo a menudo; un buen programador es siempre ordenado.
- El botón **[TOOLS] > Clippy** (Tu inspector de estilo): **Clippy** es un pequeño asistente virtual que analiza tu programa en busca de mejoras. No busca faltas de ortografía (de eso se encarga el compilador), sino que revisa si hay formas más inteligentes, modernas o eficientes de escribir lo mismo. Si pulsas Clippy desplegando el botón **[TOOLS]**, te dará sugerencias en color amarillo para que tu código sea «perfecto al estilo Rust».

4.2. El botón mágico: [SHARE]

A diferencia de cuando trabajas con un procesador de textos como LibreOffice, donde tienes que ir a Archivo ➡ Guardar y generar un fichero en tu disco duro, en la Playground no existen las carpetas. Si cierras la pestaña del navegador sin hacer nada, ¡tu código desaparecerá para siempre!

Para evitar que pierdas tus trabajos o tus juegos, Rust incluye en la barra superior un botón mágico llamado **[SHARE]** (Compartir).

Cuando tu programa funcione correctamente (o si te has encallado y necesitas ayuda), haz clic sobre el botón **[SHARE]**. Al instante, la Playground hará dos cosas de forma invisible en sus servidores:

1. Guardará una «fotografía» exacta de todo el código que tienes escrito en tu pantalla izquierda.
2. Generará una dirección web (una **URL**) única y permanente para ti.

En cuanto pulses el botón, verás que aparecen cuatro opciones en la pantalla. La única que nos interesa es la primera, llamada **Permalink to the playground** (Enlace permanente). Al lado de ella verás un icono con dos rayitas horizontales: haz clic en él para copiar la dirección web automáticamente. Ahora puedes pegar (guardar) esa dirección donde quieras.

¿Para qué sirve este enlace?

- **Para guardar tu trabajo:** Puedes copiar ese enlace y pegarlo en un documento tuyo de notas o enviártelo por correo. Cada vez que hagas clic en ese enlace, la Playground se abrirá mostrando exactamente tu código tal y como lo dejaste.
- **Para entregar tus ejercicios:** Cuando el profesor te pida una tarea, no tendrás que enviarle pesados archivos por correo ni usar un pendrive. Simplemente tendrás que enviarle ese enlace. El profesor podrá abrir tu programa en su propio ordenador, pulsar [RUN], probar tu juego y ver en qué te has equivocado para ayudarte.

📌 Nota: Si abres un enlace de SHARE antiguo, haces cambios en el código y quieres guardar la nueva versión, tendrás que volver a pulsar el botón **[SHARE]** para generar un enlace nuevo. El enlace viejo nunca se modifica; siempre se queda guardado como una foto fija en el tiempo.

4.3. Activando superpoderes: ¿Qué es un crate?

Imagínate que estás jugando a tu videojuego favorito y le instalas un mod o una expansión para tener superpoderes, coches nuevos o herramientas que el juego no traía de fábrica. En el mundo de la programación hacemos exactamente lo mismo.

El lenguaje Rust ya viene con un montón de herramientas básicas integradas (como la orden **println!** para escribir en pantalla). Pero si queremos hacer cosas más complejas o divertidas — como generar números al azar para un juego, procesar imágenes o conectar nuestro programa a internet —, no tenemos que inventar la rueda desde cero. Podemos usar piezas de código que otros programadores del mundo ya han escrito, probado y regalado a la comunidad de Rust.

En el idioma de Rust, estas piezas o paquetes de expansión se llaman **crates** (que en inglés significa «cajas de madera de mercancías», como las que caen del cielo en los juegos de supervivencia).

Al usar la **Playground web**, tenemos una ventaja increíble: los 100 crates más importantes y famosos de la historia de Rust ya están preinstalados en los servidores. No tenemos que descargar nada a nuestro ordenador. Para activar uno de estos **superpoderes** en nuestro programa, solo tenemos que usar una palabra mágica al principio de todo nuestro código: la instrucción **use** seguida del nombre del crate.

¡Vamos a probarlo! El generador de dados al azar

Vamos a pedirle a la Playground que use un crate super famoso llamado **rand** (de la palabra random, que significa «aleatorio» o al azar en inglés). Queremos crear un programa que simule el lanzamiento de un dado de 6 caras cada vez que pulsemos el botón RUN.

 Copia este código en tu Playground (limpia lo que tenías antes) y pulsa [RUN]:

Fichero: **generador_aleatorio.rs**

```
// 1. Traemos el trait 'RngExt' para activar los métodos del generador
// Y traemos el módulo raíz 'rand' para llamar a la función rand::rng()
use rand::RngExt;

fn main() {
    // 2. Le pedimos a Rust que prepare el generador de números aleatorios
    let mut generador = rand::rng();

    // 3. Lanzamos el dado: simulado con .random_range(1..7)
    let numero_dado = generador.random_range(1..7);

    println!("🎲 Has lanzado el dado y ha salido un: {}", numero_dado);
}
```

Si pulsas el botón **[RUN]** varias veces seguidas, verás que la consola de la derecha te responde con un número diferente en cada intento. ¡Acabas de crear tu primer motor de azar para un videojuego!

¿Qué ha pasado aquí?

 **Nota:** no es importante que entiendas el código. Fíjate solo en lo fácil que ha sido añadir superpoderes a nuestro programa. Una sola línea.

No obstante, aquí tienes una explicación por si te pica el gusanillo.

La primera línea, use **rand::RngExt**, es la llave que necesitamos para trabajar con el crate **rand** que utilizamos para generar números aleatorios. Este ejemplo no forma parte de lo que es la teoría de este cuaderno. Lo hemos puesto aquí simplemente para que veas como se consiguen poderes del exterior para nuestro programa.

Evidentemente no es suficiente con conseguir los poderes sino que luego hay que saber utilizarlos como se muestra para este caso en las líneas **let mut generador = rand::rng();** y **let numero_dado = generador.random_range(1..7);**.

Si borráramos la línea **use rand::RngExt;** el compilador se volvería loco y nos daría un error en rojo diciendo que no entiende qué significa eso de **.random_range(1..7)**. Gracias a los **crates** y a la **Playground**, tu capacidad para crear programas divertidos se vuelve infinita con solo una línea de código. Obviamente, repetimos, los crates están documentados y tenemos que aprender a utilizar los que queramos incluir en nuestro programa.

4.4. Lanzar el dado sin utilizar crates

El ejemplo que hemos hecho en el apartado anterior sirvió para transmitir el concepto de crate y para que veas cómo se utilizan. Solo al final del cuaderno volveremos a utilizar otro crate e indicaremos cómo hacerlo.

Simular mediante un programa el lanzamiento de un dado puede hacerse también sin utilizar ningún crate externo, solo con las instrucciones que incorpora Rust como podemos ver en el siguiente programa.

 Copialo, pegalo en la playground y ejecútalo.

Fichero: **lanzar_dado.rs**

```
fn main() {
    // Genera un número aleatorio entre 1 y 6 directamente (el 7 no se incluye)
    let numero_dado = rand::random_range(1..7);

    println!("🎲 Has lanzado el dado y ha salido un: {}", numero_dado);
}
```

4.5. Aprendiendo a leer el «idioma» del compilador

Guía visual para no asustarse con los mensajes en rojo (errores) y amarillo (consejos de estilo); aprendiendo a buscar la línea exacta donde está el fallo.

Cuando pulsas el botón de **RUN**, la Playground **compila** primero el programa y a continuación, si no han habido errores de compilación, lo **ejecuta**.

El compilador no es un enemigo que juzga tu código; es un asistente automatizado con un detector de errores ultra-sensible. Cuando tu código no compila o muestra avisos, no significa que hayas «roto» la computadora. Simplemente significa que el compilador ha encontrado algo que no entiende o que podría hacerse mejor.

Aprender a leer su «idioma» te ahorrará horas de frustración.

 **La regla de oro: Errores vs. Advertencias (Warnings)** Lo primero que debes aprender a distinguir es la gravedad del mensaje. Los compiladores modernos utilizan un código de colores estándar:

Color	Tipo de Mensaje	¿Qué significa?	¿Detiene la ejecución?
-------	-----------------	-----------------	------------------------

Rojo	Error de Compilación	El código tiene un fallo sintáctico o lógico grave. El compilador no entiende qué quieres hacer.	SÍ. No se generará el programa ejecutable hasta que lo arregles.
Amarillo	Advertencia (Warning)	El código es válido y se puede ejecutar, pero es un consejo de estilo o has realizado una práctica peligrosa (ej. una variable que creaste pero nunca usas).	NO. El programa funcionará, pero ignorarlo puede traer «bugs (errores)» ocultos.

Tu plan de acción ante un error (mensaje en rojo)

Cuando te enfrentes a una lista enorme de errores, sigue este protocolo para mantener la calma:

- **Paso 1: Ve siempre al primer error.** Un solo punto y coma olvidado al principio del archivo puede desencadenar 50 errores falsos en las líneas siguientes porque el compilador se desorienta. Arregla el primero de la lista y vuelve a compilar.
- **Paso 2: Copia y pega buscando ayuda.** Si no entiendes la descripción del error, no adivines. Copia el texto explicativo (ejemplo: `error: y1_r0 holds a non-trivially copyable type`) y búscalo en Google o StackOverflow. Alguien ya se equivocó en eso mismo antes que tú. Y siempre puedes preguntar a alguna **IA** por el error. Indícale que estás utilizando Rust, copia el primer error y mándalo al chat.

4.6. Tu copiloto digital: Cómo preguntar a la IA

El compilador de Rust (**rustc**) es famoso por ser uno de los más descriptivos y serviciales del mundo. Sin embargo, cuando estás empezando, sus explicaciones detalladas pueden resultar abrumadoras. Es en ese momento cuando una Inteligencia Artificial (**IA**) puede convertirse en tu mejor tutor personal, siempre y cuando sepas cómo comunicarte con ella.

El objetivo no es que la IA te dé el código ya resuelto para copiar y pegar, sino usarla como un puente para entender qué te está pidiendo el compilador.

El método correcto: Copiar desde la Rust Playground.

Cuando tu código falla en la Rust Playground, la terminal inferior te mostrará el error estructurado. Para pedir ayuda de forma eficiente, debes proporcionar a la IA el **contexto completo**.

Sigue estos pasos para preparar tu consulta:

- Copia el código completo de tu programa en la ventana de edición.
- Copia el mensaje de error íntegro de la consola inferior (incluyendo los bloques que dicen `help:` o `note:`, ya que Rust suele incluir ahí la solución exacta).
- Pega finalmente los contenidos anteriores en la ventana de chat con la IA como parte final de tu prompt (consulta, pregunta).

Fórmulas de Prompts: Cómo pedir explicaciones (y no solo respuestas).

Si le dices a la IA «Arréglame este código», te devolverá un bloque para copiar que funcionará, pero no habrás aprendido nada y volverás a cometer el mismo error cinco minutos después.

Para obligar a la IA a actuar como un profesor y no como un generador de código automático, utiliza estas plantillas de instrucciones (prompts):

- **Para entender el concepto:**

«Estoy aprendiendo Rust en la Playground. Mi código da este error de compilación. No me des el código corregido aún. Explícame en un lenguaje sencillo qué regla de Rust estoy rompiendo y qué significa este error.»

- **Para buscar pistas:**

«Este es mi código y este es el error de Rust. Dame una pista o una guía paso a paso de qué debo modificar en mi lógica, pero no me escribas las líneas de código finales.»

- **Para corregir y contrastar (Auto-evaluación):**

“No logro solucionar este error en Rust. Por favor, muéstrame el código corregido y explica detalladamente qué cambiaste línea por línea respecto a mi versión original.»

4.7. Las 3 reglas para no volverte dependiente de la IA

Para que tu aprendizaje en Rust sea sólido, grábate estas tres reglas cuando uses herramientas de IA:

- **Regla 1: Lee primero la sugerencia de Rust.** Antes de saltar a la IA, lee la sección help: que te da la Playground. El 80% de las veces, Rust te dice exactamente qué carácter añadir o qué función cambiar. Entrenar tu ojo para leer al compilador es tu superpoder como programador.

- **Regla 2: Prohibido el «Copiar y Pegar» a ciegas.**

Si la IA te da una línea corregida, no la pegues sin más. Tecléala tú mismo en la Playground. El acto físico de teclear el código ayuda a tu cerebro a fijar la sintaxis (especialmente con los estrictos operadores de Rust como `&`, `*` o `mut`).

- **Regla 3: El validador final es la Playground, no la IA.**

Las IA a veces inventan métodos o sintaxis que no existen en Rust (alucinaciones). Si la IA te dice una cosa y la Rust Playground te sigue dando rojo, la Playground siempre tiene la razón. Trust the compiler.

5. Laboratorio

Como vamos a trabajar en la **Rust Playground**, para probar un programa podrás copiarlo del cuaderno y pegarlo en la parte izquierda de la ventana de la Playground. No obstante, si te has descargado el repositorio de los cuadernos también tienes allí los mismos códigos guardados en ficheros con el mismo nombre que se indica en el cuaderno. Puedes también copiar los códigos desde los ficheros y pegarlos en la playground.

5.1. Sección 1: Despegue

A partir de esta sección empezaremos a programar y explicar distintos ejemplos para que consolides lo que has aprendido en teoría y te diviertas modificando y ejecutando los ejercicios que proponemos.

5.1.1. El programa hola mundo

El siguiente programa debe presentar en la pantalla el texto: **¡Hola, Mundo desde la Playground!**

- Lee los comentarios y ejecuta el programa pulsando el botón **[RUN]** o **[CONTROL] + [ENTER]** en tu Playground.
- Sustituye el mensaje que se imprime por otro mensaje alternativo de tu elección y vuelve a ejecutar el programa.

Fichero: **hola_mundo.rs**

```
// Bienvenidos a vuestro primer programa en Rust.
// Las dos barras // sirven para escribir notas
// que el ordenador ignora.

fn main() {
    // Todo programa en Rust empieza a ejecutarse dentro de estas llaves { }

    println!("¡Hola, Mundo desde la Playground!");

    // 💡 El punto y coma ';' al final indica que la orden ha terminado.
}
```

A simple vista parece poca cosa, pero aquí dentro se esconden las tres reglas sagradas de la estructura de Rust que debes memorizar:

1. La Llave de Entrada al programa, la función **fn main()**:

La palabra **fn** significa **función** (un bloque de código que hace una tarea). La palabra **main** significa **principal** en inglés. En Rust, todo, absolutamente todo lo que quieras que tu ordenador ejecute, debe vivir dentro de la función principal. Es la puerta de acceso a la memoria. Más tarde veremos que pueden también existir funciones que se escriben fuera de la función **main**, arriba o abajo de la misma.

2. El Contenedor (Las llaves **{ }**):

Las llaves actúan como las paredes de una habitación. La llave de apertura `{` marca dónde empieza a trabajar el ordenador y la llave de cierre `}` marca el final del camino. Si dejas una habitación sin una de sus paredes o las dos, el programa se romperá.

3. El Emisor de Píxeles, `println!("texto")`:

Esta orden (que se pronuncia print-line) toma el texto que escribas entre comillas y lo estampa en la pantalla negra de la derecha.

Veamos algunos detalles importantes:

A) El punto y coma ; y los comentarios

Si miras con atención el código anterior, notarás dos detalles cruciales: el punto y coma final y el uso de anotaciones.

- **El cierre de orden con “;”:**

Cada vez que le das una instrucción de acción al ordenador, debes terminar la línea con un punto y coma. Piensa en él como el punto final de una frase en un libro. Si lo omites, el ordenador se quedará esperando y no sabrá dónde termina una orden y dónde empieza la siguiente. Solo algunas líneas muy concretas en los programas no se finalizan con punto y coma. Obsérvalo en los listados para que no te pase desapercibido.

- **Los comentarios (//):**

Si en cualquier parte de tu pantalla escribes dos barras seguidas `//`, todo lo que escribas a continuación en esa misma línea incluidas las dos barras se volverá de color gris. El ordenador ignorará por completo ese texto.

Sirve para que tú, como programador, dejes notas explicativas para ti mismo o para tus compañeros.

B) Las reglas del juego: ¿Qué pasa cuando rompemos el código?

Programar consiste en experimentar, equivocarse y aprender a corregir.

Si intentas ejecutar el siguiente código en la Playground obtendrás errores que en el listado hemos señalado con dos comentarios que empiezan por una cruz roja **✗** ¡peligro!:

```
fn main() {
    println!("Esto dará un fallo") // ✗ <--- error, no funcionará

    print_pantalla!("Hola"); // ✗ <--- error, no funcionará
}
```

✗ Control de Daños, ¿en qué consiste?:

Cuando un programa no funcione debido a un error en rojo del compilador, tu red de seguridad inmediata es comentar la línea que produce el error. El compilador te indicará el número de la línea. Añade `//` al principio de la instrucción que falla. Al hacerlo, la línea se volverá gris, el compilador la pasará de largo y te permitirá ejecutar el resto del programa (si es posible sin esa línea) para comprobar que lo demás está bien.

5.1.2. Qué hacer cuando obtenemos errores del compilador

En el apartado anterior aprendiste la teoría del «control de daños», y ahora vamos a ponerla en práctica. Si intentas pulsar **[RUN]** en la Playground con el código de abajo tal cual está, verás que la pantalla se tiñe de rojo. El compilador se ha negado a compilar el programa.

Vamos a jugar a los detectives para entender qué está pasando aquí dentro analizando el programa:

Fichero: **destripando_main.rs**

```
fn main() {
    // En Rust, la orden para mostrar texto en pantalla lleva un signo de
    // exclamación: println!()
    println!("Esto funciona perfectamente");

    // ¿Qué pasa si olvidamos el punto y coma al terminar la orden?
    println!("Esto dará un fallo") // ✗ <--- error, no funcionará

    // ¿Qué pasa si escribimos mal el nombre de la función?
    print_pantalla!("Hola");      // ✗ <--- error, no funcionará
}
```



Experimento 1: El punto y coma fantasma

Fíjate en la consola de errores de la Playground. El compilador se queja en la línea 6. Te dice algo como: **help: add ; here**. Debes añadir un punto y coma donde te indica. Como no pusiste el punto y coma, Rust se ha desorientado y piensa que la orden se ha fusionado con lo que viene después.

Vamos a copiar las líneas que describen el error tal cual las entrega el compilador:

```
Standard Error

Compiling playground v0.0.1 (/playground)
error: expected `;`, found `print_pantalla`
--> src/main.rs:6:35
|
6 |     println!("Esto dará un fallo") // ✗ <--- error, no funcionará
|                                     ^ help: add `;` here
...
9 |     print_pantalla!("Hola");      // ✗ <--- error, no funcionará
|     ----- unexpected token

error: cannot find macro `print_pantalla` in this scope
--> src/main.rs:9:5
|
9 |     print_pantalla!("Hola");      // ✗ <--- error, no funcionará
|     ^^^^^^^^^^^^^^^^^^^^^^

error: could not compile `playground` (bin "playground") due to 2 previous errors

Standard Output
```

Figura 1: *Error: expected ;, found print_pantalla*

- **Misión:** Aplica el control de daños. Añade dos barras `//` al principio de la línea 6 para que se vuelva gris. Vuelve a pulsar [RUN]. El compilador seguirá marcandote el error de la línea 9.

♂ Experimento 2: El idioma inventado

Al comentar la línea 6, el compilador por fin puede avanzar... ¡pero vuelve a protestar! Ahora el error se desplaza a la línea 9. El compilador te dice error: cannot find macro `print_pantalla` in this scope. Rust es muy estricto con las palabras sagradas; si inventas una orden que no existe en su diccionario (como `print_pantalla!` en lugar de `println!`), la máquina se detiene porque no sabe qué instrucciones enviar a la pantalla.

- **Misión:** Corrige la línea 9 cambiando `print_pantalla!` por `println!`. Quita también las barras `//` de la línea 6 y ponle el punto y coma `;` que le faltaba al final de la línea. ¡Vuelve a compilar y disfruta de tu pantalla en verde!

5.1.3. Tipos e inferencia

Vamos a trabajar con el código del fichero que indicamos un poco más abajo.

- La instrucción: `let edad: i32 = 14;` significa que declaramos la variable `edad` como entera de 32 bits y la inicializamos con el entero 14.
- La instrucción `let nombre: &str = "Halcón68";` significa que declaramos la variable `nombre` como texto y la inicializamos con la cadena de texto "Halcón68" sin las comillas.

Fichero: `tipos_e_inferencia.rs`

```
fn main() {
    // 1. Declaración Explícita (Le decimos a Rust exactamente qué tipo es)
    // i32 significa: Número Entero de 32 bits (números sin decimales)
    let edad: i32 = 14;

    // &str significa: Cadena de texto (letras entre comillas)
    let nombre: &str = "Halcón68";

    println!("Alumno: {}, Edad: {} años", nombre, edad);

    // Intentar guardar un texto en una caja reservada para números:
    let puntos: i32 = "diez"; // ✗ <--- error, no funcionará

    // 2. La Magia de la Inferencia (Rust es listo y adivina el tipo)
    let nivel = 1; // Rust sabe automáticamente que es un número entero (i32)
    let lenguaje = "Rust"; // Rust sabe automáticamente que es texto (&str)

    println!("Estudias el nivel {} de {}", nivel, lenguaje);
}
```

 Copia y pega el programa anterior en la Playground. Ejecuta el programa pulsando sobre el botón [RUN]. Se producirá el siguiente error en la playground:

```
--> src/main.rs:12:23
|
12 |     let puntos: i32 = "diez"; // ✗ <--- error, no funcionará
```

```
|
|      --- ^^^^^^ expected `i32`, found `&str`
|      |
|      expected due to this
```

Efectivamente. Declaraste la variable **puntos** como entera de 32 bits, **let puntos: i32 = "diez";** pero le asignaste el texto "diez". Rust es muy estricto con los tipos. Si declaras una variable de un tipo, en el ejemplo **i32**, luego no puedes asignar a esa variable un valor de otro tipo, por ejemplo de tipo **Texto** como ocurre en el programa. Por eso el compilador generó el error.

Comenta esa línea con dos `//` barras al principio y vuelve a ejecutar con [RUN] para obtener los resultados esperados. Debes ver la línea así: (`// let puntos: i32 = "diez";`).

En la programación, guardar información es como organizar una mudanza: necesitas cajas de cartón. Rust es increíblemente ordenado y exige saber qué tipo de objeto va dentro de cada caja para que nada se rompa por el camino.

Ejecuta nuevamente este código en tu Playground para ver cómo gestiona Rust sus «cajas» (a las que llamamos **variables**).

A continuación te proponemos **repetir de nuevo los pasos anteriores** para que interiorices bien este proceso de detección y comprensión de errores y tengas alguna herramienta para poder (cuando sea posible) anular el error y que el programa siga ejecutándose hacia abajo en el código.

Experimento 1: El conflicto de las cajas

Si pulsas [RUN] tal como estaba el código al principio, la Playground se detendrá en la línea **let puntos: i32 = "diez";** donde intentamos crear los puntos. El compilador te lanzará un error cristalino al principio con letras blancas en fondo rojo: **mismatched types (tipos que no coinciden)**.

Le has dicho a Rust: Voy a crear una caja para números enteros **i32** pero luego has intentado meter dentro la palabra **diez** escrita con letras **&str**. Rust se niega en redondo a mezclar churras con merinas para evitar que el programa falle en el futuro.

- **Misión:** Aplica el control de daños. Comenta la línea que falla añadiendo `//` al principio para que se vuelva gris. Vuelve a pulsar [RUN].

 Los dos conceptos sagrados de este código:

1. **Las etiquetas de las cajas (let):** En Rust, cada vez que quieres crear una variable nueva, tienes que usar la palabra mágica `let`. Piensa en ella como la orden de «fabricar una caja nueva en la memoria».
2. **La Magia de la Inferencia:** Fíjate en las variables **nivel** y **lenguaje**. ¡No les hemos puesto los dos puntos : ni el tipo de datos! Rust tiene un cerebro digital tan potente que lee el valor de la derecha, ve un `1` y deduce él solo: «Vale, esto es un número entero, no hace falta que el programador me lo especifique», prepararé una caja `i32` para enteros. Luego en esta caja no puedes meter un texto.
3. A este superpoder de adivinar el tipo se le llama Inferencia de tipos y solo es posible cuando en una asignación con el operador `=`, el tipo de la parte derecha del igual es cococado y se le asigna a la parte izquierda del igual.

5.1.4. Mis primeras funciones

Vamos a poner en práctica la teoría del «divide y vencerás». En este experimento, a continuación de esta explicación teórica, vamos a crear tres funciones propias fuera del `main` y vamos a darle órdenes personalizadas al ordenador. Es decir, le diremos al `main` que ejecute esas funciones

📌 Nota: Las funciones se escriben fuera del `main` y pueden estar tanto arriba como abajo del `main`.

En primer lugar vamos a describir desde un punto de vista más teórico tres conceptos clave:

1. **Qué es una función**
2. **Qué son los parámetros -y sus tipos asociados- de una función**
3. **Que son los argumentos pasados a los parámetros de una función**

Vamos a hacer el razonamiento sobre un pequeño programa de ejemplo que explicaremos prácticamente línea a línea: Es el siguiente:

```
fn sumar_numeros(a: i32, b: i32) {
    let resultado = a + b;
    println!("La suma de {} + {} es: {}", a, b, resultado);
}
fn main() {
    sumar_numeros(3, 4); // Pasamos los ingredientes correctos
}
```

📌 Nota: La función `sumar_numeros` acepta dos numeros `a` y `b`, calcula la suma, la coloca en la variable `resultado` y finalmente imprime `a`, `b` y `resultado` en los huecos `{}`.

Lo primero que debemos observar es que la función, denominada **`sumar_numeros`**, se define fuera de la función **`main`** mediante este código:

```
fn sumar_numeros(a: i32, b: i32) {
    let resultado = a + b;
    println!("La suma de {} + {} es: {}", a, b, resultado);
}
```

La primera línea utiliza **`fn`** para definir el nombre de la función: **`sumar_numeros`**, sus dos parámetros: **`a`** y **`b`** y los tipos de esos parámetros que son enteros **`i32`**.

La potencia de las funciones se observa cuando se llaman desde la función **`main`** o desde cualquier otra función. En el ejemplo, nosotros llamamos a la función **`sumar_numeros`** en la única línea que tiene nuestra función **`main`**:

```
sumar_numeros(3, 4);
```

¿Qué pasa cuando se ejecuta la línea de arriba? Vayamos por partes:

1. El programa **abandona la ejecución del main** en esa misma línea y **ejecuta** la función **sumar_numeros**.
2. Para ello, copia en el parámetro **a** de la función el argumento **3** que le pasamos en la línea anterior (la llamada a la función) y en el parámetro **b** el argumento **4**.
3. A continuación ejecuta las dos líneas del cuerpo de la función pero tomando **a = 3** y **b = 4**. Primero realiza la suma y luego imprime por pantalla.
4. Finalmente, el programa abandona la función **sumar_numeros** y regresa a la línea siguiente desde la que salto, es decir, a la línea siguiente de **sumar_numeros(3, 4)**; en la función **main** y sigue hasta que la función **main** termine. En realidad, nosotros ya no tenemos nada en el **main** después de **sumar_numeros(3, 4)**; luego cuando el programa regresa al **main** después de haber ejecutado la función **sumar_numeros**, inmediatamente termina.
5. Es posible que una función no tenga parámetros como la función **saludar_alumno** del siguiente ejemplo, en cuyo caso, tampoco se le pasan argumentos al llamarla.

Como vemos, el programa realiza un viaje de ida y vuelta a la función y realiza una tarea (en este caso sumar dos números e imprimir el resultado) sin que el código para realizar la suma esté escrito dentro de la función **main**, está en el cuerpo de la función **sumar_numeros**.

Al definir la función hemos definido los parámetros **a** y **b** como números enteros (**a: i32**, **b: i32**) y luego le hemos pasado argumentos enteros al llamar la función con **sumar_numeros(3, 4)**; hemos pasado **a = 3** y **b = 4** y por eso funciona. Si hiciéramos la llamada **sumar_numeros(3.5, 4)**; el programa se rompería y nos arrojaría un error. El argumento **a** no puede ser decimal.

Veamos un ejemplo un poco más completo en acción.

 Copia este código en tu Playground:

Fichero: **mis_primeras_funciones.rs**

```
// 1. Fabricamos nuestra primera función propia
// Esta función no necesita parámetros, solo ejecuta una acción visual
fn saludar_alumno() {
    println!("👋 ¡Hola, Alumno de Rust!");
    println!("🚀 Bienvenido a tu zona de entrenamiento.");
}

// 2. Fabricamos una función que hace un cálculo matemático
// Le pedimos que nos multiplique un número por 2
fn duplicar_numero(numero: i32) {
    let resultado = numero * 2;
    println!("🧮 El doble de {} es: {}", numero, resultado);
}

// 3. Una función con DOS ingredientes para sumar
fn sumar_numeros(a: i32, b: i32) {
    let resultado = a + b;
    println!("➕ La suma de {} + {} es: {}", a, b, resultado);
}

fn main() {
```

```
println!("🚩 El programa principal (main) se ha iniciado.\n");

// 💡 LLAMADA A LAS FUNCIONES: Aquí despertamos a nuestras recetas
saludar_alumno(); // El ordenador salta al código de la función

println!("\n--- Haciendo cálculos en el laboratorio ---");

duplicar_numero(10); // Le pasamos el número 10 como ingrediente
duplicar_numero(50); // Reutilizamos la misma función con otro dato

println!("\n🏁 El programa principal va a terminar.");
sumar_numeros(3, 4); // Pasamos los ingredientes correctos

// ❌ Error provocado para el laboratorio:
// sumar_numeros(3.2, 4);
}
```

⚙️ Experimento 1: El salto del ordenador

Si pulsas **[RUN]**, verás que el texto de salida en la pantalla aparece perfectamente ordenado. El ordenador **entra en el main**, imprime la bandera de salida, y al leer **saludar_alumno()**; detiene lo que está haciendo, «viaja» arriba a donde empieza la función, ejecuta los dos saludos y vuelve a **main** para seguir con la siguiente línea. La función **saludar_alumno()** no tiene parámetros por lo tanto, no recibe argumentos en la llamada.

- **Misión:** Intenta duplicar la línea **saludar_alumno()**; en el main para que aparezca dos veces seguidas. Vuelve a pulsar **[RUN]**. Verás cómo el saludo se repite en pantalla sin necesidad de que hayas tenido que volver a teclear los dos **println!** originales. ¡Has ahorrado código!

🧠 Los «ingredientes» de una función

Fíjate en la función **duplicar_numero(numero: i32)**. Lo que hay dentro de sus paréntesis (**numero: i32**) es su ingrediente (en programación lo llamamos parámetro). Le estamos diciendo a Rust: «Para que esta función realice su trabajo, necesita que le den un número (argumento) entero (i32)». Al llamarla desde el main con **duplicar_numero(10)**;, la función atrapa el 10 (el argumento) y lo asigna al parámetro **a**, lo multiplica por 2 y nos muestra el resultado en la pantalla de forma limpia.

⚙️ Experimento 2: El diccionario de la programación y el guardián de la puerta

Si miras la última función añadida, **fn sumar_numeros(a: i32, b: i32)**, descubrirás cómo hablan los programadores profesionales. Es un lenguaje técnico muy sencillo de entender:

- Los **parámetros** (**a: i32, b: i32**): Son las variables que escribimos en la definición de la función. Son los «moldes» o los «huecos reservados». Le dicen a Rust: «Para poder sumar, necesito que me prepares dos huecos para números enteros».
- Los **argumentos** (**3 y 4**): Son los valores reales y vivos que le pasamos a la función cuando la llamamos dentro del **main()**. El 3 se mete en el hueco de la **a** y el 4 en el de la **b**.

🔴 Rompiendo el guardián de Rust (Prueba de laboratorio)

Quita las dos barras `//` de la última línea (`sumar_numeros(3.2, 4);`) y pulsa **[RUN]**.

- **El resultado:** La Playground se volverá a teñir de rojo con un error de **mismatched types**. El compilador te dirá que esperaba un entero `i32` pero le has enviado un decimal flotante (un decimal, a todos los efectos).
- **La lección:** En Rust, las funciones tienen un guardián ultra-estricto en la puerta. Si la receta pide un número entero sin partir (`i32`), no puedes intentar colarle un decimal con tropezos (`3.2`). Rust prefiere romper el programa antes de permitir que una función cocine con los ingredientes equivocados.
- **Corrección:** Vuelve a poner las barras `//` para dejar el laboratorio funcionando en verde.

Lo mismo ocurriría si la función espera un valor booleano (`false` o `true`) en uno de sus parámetros y le pasamos un entero `i32`.

5.2. Sección 2: Cajas y permisos

Para poder manejar información, el programa debe acceder a los lugares de la memoria en donde está almacenada. Vamos a hacer un símil entre variables del ordenador que almacenan la información y cajas que contienen información con permisos para que esa información **solo pueda ser vista** (leída) o por contra, para que **pueda ser vista y modificada** (reescrita).

5.2.1. La palabra `let` para variables inmutables

En otros lenguajes de programación, cuando creas una caja (variable) puedes cambiar lo que hay dentro siempre que quieras. En Rust, las cosas son diferentes. Rust es un lenguaje diseñado para ser ultra-seguro, y para evitar accidentes aplica una regla muy estricta: **todas las cajas nacen cerradas con llave**. Cuando creas una variable con `let`, por ejemplo en el código de abajo con `let puntuacion = 10;` posteriormente en el programa ya no podrás cambiar el valor de la variable `puntuacion`. Veremos que hay otra forma de proceder en la que si se puede cambiar el valor de las variables después de haberles asignado el primer valor.

 Ejecuta este código en tu Playground y mira qué pasa cuando intentas romper esa regla:

Fichero: **cajas.rs**

```
fn main() {
    // En Rust, cuando creamos una variable (una caja) es como cerrarla con un
    // candado.
    let puntuacion = 10;
    println!("Tu puntuación inicial es: {}", puntuacion);

    // Intentamos cambiar el valor de la caja cerrada:
    puntuacion = 20; // ✗ <--- error, no funcionará

    // El ordenador no te dejará ejecutar el programa porque por defecto las
    // variables
    // son "inmutables" (no se pueden cambiar) para evitar accidentes.
}
```

Experimento 1: El candado de Rust

Al pulsar **[RUN]**, la Playground se detendrá en seco en la línea donde intentas cambiar la puntuación a 20. El compilador te mostrará un **mensaje de error** que dice algo como: `cannot`

assign twice to immutable variable (**no se puede asignar dos veces a una variable inmutable**).

Para Rust, la palabra **Immutable** significa «prohibido tocar tras su fabricación». Si tú creas con **let** una caja con un 10 dentro, ese valor se queda congelado para siempre.

- **Misión:** Aplica el control de daños que ya conoces. Añade las dos barras `//` al principio de la línea que da error (`puntuacion = 20;`). Vuelve a pulsar **[RUN]** para comprobar que el programa compila en verde y muestra el mensaje inicial en la pantalla.

💡 ¿Por qué hace esto Rust?

Imagina que estás programando un videojuego gigante y creas una variable para la velocidad máxima del personaje. Si cualquier parte del código pudiera cambiar esa variable por accidente, el juego podría volverse loco. Al poner un candado por defecto (**let**) a esta variable, Rust se asegura de que nadie cambie datos importantes sin tu permiso explícito.

5.2.2. Las palabras `let` `mut` para variables mutables

¿Y qué pasa si realmente necesitamos cambiar el valor de una caja? Por ejemplo, las vidas de un jugador, la puntuación o el nivel actual. Para esos casos, Rust nos da una llave especial: la palabra mágica **mut** (abreviatura de mutable, que significa «capaz de cambiar»).

📄 Copia este código en tu Playground y observa cómo cambia el comportamiento del compilador.

Fichero: **mutacion.rs**

```
fn main() {
    // Al añadir la palabra 'mut', le ponemos una etiqueta de "permitido
    cambiar" a la caja.
    let mut vidas = 3;
    println!("Empiezas la partida con {} vidas.", vidas);

    // Como la caja tiene el permiso 'mut', ahora sí podemos modificar su
    contenido:
    vidas = 2;
    println!("¡Te ha tocado un enemigo! Ahora te quedan {} vidas.", vidas);

    // ⚠️ ¡Cuidado! Puedes cambiar el valor, pero NO el tipo de dato que guarda
    la caja:
    vidas = "cero"; // ❌ <--- error, no funcionará

    // Explicación: La caja 'vidas' se creó para guardar números enteros.
    // No puedes meter texto dentro de una caja de números.
}
```

⚙️ Experimento 1: La mutación exitosa y el límite de Rust

Si pulsas **[RUN]**, verás que la Playground se tiñe de rojo al llegar a la línea donde pone **vidas = "cero";**. El compilador te lanzará otra vez el error de `mismatched types` (tipos que no coinciden). Intentaste colocar el texto "cero" en la variable **vidas** que por inferencia de tipos es de tipo `i32`. La inferencia se produjo en la línea **let mut vidas = 3;**.

- **Misión:** Aplica tu técnica de control de daños. Comenta la línea donde intentas meter la palabra «cero» en la variable **vidas** escribiendo `//` al principio. Vuelve a pulsar **[RUN]** para ver el programa funcionar de principio a fin de forma impecable.

Ahora el programa consigue ejecutar las dos primeras órdenes de texto sin problemas. ¡Hemos logrado cambiar las vidas de **3 a 2** gracias a **let mut**!

La lección clave: Permiso para cambiar, no para transformarse

Este experimento nos enseña una de las reglas más importantes de Rust:

- **mut** te da permiso para cambiar el contenido de la caja (el número 3 por el número 2)
- **mut NO** te da permiso para cambiar la naturaleza de la caja. Si la caja nació para guardar números enteros, morirá guardando números enteros. No puedes transformarla de repente en una caja de texto.

5.2.3. Prácticas sobre permisos en el laboratorio

Para terminar esta sección, vamos a simular el motor de un videojuego real. Aquí vas a ver cómo un programador decide, antes de empezar, qué cajas deben llevar candado y cuáles deben quedarse abiertas con **mut**. Además, nos enfrentaremos a un error muy común: los despistes al teclear.

 Copia este código en tu Playground y prepárate para resolver el misterio.

Fichero: **operaciones_laboratorio.rs**

```
fn main() {
    // Datos fijos del juego (no necesitan cambiar, van sin 'mut')
    let nombre_jugador = "Halcón68";
    let puntos_por_enemigo = 150;

    // Datos dinámicos del juego (van a cambiar, necesitan 'mut')
    let mut puntuacion_total = 0;
    let mut nivel_actual = 1;

    println!("--- ESTADÍSTICAS DE {} ---", nombre_jugador);
    println!("Nivel: {} | Puntos: {}", nivel_actual, puntuacion_total);

    // El jugador derrota a un enemigo: aumentamos la puntuación

    // ⬇️ ❌ --- error, no funcionará
    puntuacion_total = puntuacion_total + points_por_enemigo;
    // (Nota: ¡Ojo con el inglés! La variable se llama 'puntos_por_enemigo')

    // Corrección del error (sustituye la línea errónea de arriba por esta):
    // puntuacion_total = puntuacion_total + puntos_por_enemigo;
    nivel_actual = 2;

    println!("--- ¡ENEMIGO DERROTADO! ---");
    println!("Nivel: {} | Puntos Totales: {}", nivel_actual, puntuacion_total);
}
```

Experimento 1: El error de traducción

Si pulsas [RUN], la Playground se detendrá en la línea inferior al comentario con un cruz en rojo. El compilador te dirá: **cannot find value points_por_enemigo in this scope** (no se encuentra el valor en este ámbito).

¿Qué ha pasado? La variable mutable **puntuacion_total** está bien escrita, pero al hacer la suma hemos escrito **points_por_enemigo** (en inglés) en lugar de **puntos_por_enemigo** (en español). Para el ordenador, cambiar una sola letra es como hablarle en un idioma totalmente diferente.

- **Misión** (Arreglar en lugar de comentar): Esta vez no vamos a usar el control de daños de ocultar la línea con `//`. Vamos a arreglarla de verdad. Cambia la palabra **points_por_enemigo** por **puntos_por_enemigo**. Vuelve a pulsar [RUN] y observa cómo el juego calcula la puntuación y te sube de nivel con éxito.

La lección de diseño: ¿Cuándo usar mut?

Fíjate en la estrategia que hemos usado: **nombre_jugador** y **puntos_por_enemigo** no cambian durante la partida. Se quedan **sin mut**. Son seguros y eficientes. **puntuacion_total** y **nivel_actual** se están recalculando constantemente. Necesitan **mut** obligatoriamente.

A partir de ahora, cada vez que crees una variable, hazte la pregunta mágica: ¿Este dato va a cambiar a lo largo del programa? Si la respuesta es no, no le pongas mut.

5.3. Sección 3: Tomar decisiones en el código (if, else if, else)

Para que un programa sea inteligente y dinámico, debe ser capaz de tomar decisiones basadas en diferentes condiciones. Aquí es donde entran en juego las estructuras condicionales **if**, **else if** y **else**. Estas herramientas nos permiten evaluar si una condición es verdadera o falsa y, a partir de ahí, decidir qué bloques de código se deben ejecutar y cuáles se deben ignorar.

Es importante destacar que estas estructuras son completamente modulares y adaptables a tus necesidades:

- El **if** es el único elemento obligatorio para abrir un bloque condicional.
- El **else if** no es obligatorio, y puedes usar tantos como necesites para evaluar múltiples condiciones alternativas.
- El **else** tampoco es obligatorio; sirve únicamente como un «salvavidas» final para ejecutar código cuando ninguna de las condiciones anteriores se ha cumplido.

5.3.1. Tomar decisiones en base a preguntas en el código

A continuación analizamos un ejemplo.

 Copia este código en tu Playground y lee la explicación del código

Fichero: **tomar_decisiones.rs**

```
fn main() {
    // Definimos una variable de ejemplo
    let puntuacion = 85;

    // 1. Un "if" en solitario (completamente válido)
    if puntuacion >= 50 {
        println!("¡Has aprobado el examen!");
    }
}
```

```

    }

    // 2. Una estructura completa con "else if" y "else"
    if puntuacion >= 90 {
        println!("Excelente: Tienes una A.");
    } else if puntuacion >= 80 {
        println!("Muy bien: Tienes una B.");
    } else if puntuacion >= 70 {
        println!("Bien: Tienes una C.");
    } else {
        println!("Necesitas mejorar tu nota.");
    }
}

```

💡 Explicación del código:

- **Sintaxis limpia:** A diferencia de otros lenguajes, en Rust las condiciones del `if` y `else if` no llevan paréntesis (es decir, escribimos `if puntuacion >= 50` en lugar de `if (puntuacion >= 50)`). Sin embargo, las llaves `{}` son siempre obligatorias, incluso si el bloque de código solo tiene una línea como en los casos anteriores.
- **Evaluación booleana estricta:** La condición que evalúa el `if` debe ser estrictamente un valor booleano (`true` o `false`). Rust no permite «valores de verdad» implícitos (por ejemplo, no puedes usar un número entero directamente como condición).
- **El flujo del programa:** En la primera parte, el programa comprueba si **puntuacion es mayor o igual a 50**. Como 85 lo es, imprimirá el mensaje **¡Has aprobado el examen!**. En la segunda parte, irá evaluando en orden las condiciones de arriba a abajo. Al llegar a `else if puntuacion >= 80`, la condición se cumple (`true`), por lo que imprimirá **Muy bien: Tienes una B.** e ignorará el resto de opciones (`else if >= 70` y `else`).

5.4. Sección 4: El bucle for con rangos

Imagina que le pides a un ordenador que pinte una línea de 100 píxeles en la pantalla. No vas a escribir la instrucción de pintar un píxel 100 veces, ¿verdad? Para evitar esa repetición absurda existen los bucles.

En Rust, la herramienta más potente y limpia para repetir una tarea un número exacto de veces es el bucle `for` combinado con rangos:

```

for x in 0..N {
    // El código que quieres repetir se escribe aquí
}

```

¿Cómo funciona exactamente?

- **0..N (El Rango):** Es una máquina de generar números consecutivos. Empieza en el primer número (0) y genera el siguiente en cada vuelta del bucle. El operador `..` es exclusivo, lo que significa que llegará hasta el número anterior a N. Por ejemplo, `0..5` generará los números 0, 1, 2, 3 y 4 (un total de 5 vueltas).
- **x (La Variable de Control):** Es una variable temporal que «atrapa» el número que toca en cada vuelta. En la primera vuelta x vale 0, en la segunda vale 1, y así sucesivamente.

- **in:** Es la palabra clave que le dice al bucle de dónde tiene que sacar los valores (en este caso, de nuestro rango).

¿Para qué sirven en el mundo real?

1. **Contar y repetir:** Hacer que una acción se ejecute exactamente N veces.
2. **Recorrer colecciones por su posición:** Si tienes una **lista** de datos (como un array), puedes usar **x** como el índice (la posición) para examinar o modificar lo que hay dentro de **lista[x]**.
3. **Movernos por coordenadas (Imágenes y Mapas):** Como los píxeles de una imagen se organizan en filas y columnas numeradas (desde el píxel 0 hasta el ancho de la imagen), usamos estos bucles para decirle al programa: «Ve al píxel x, procesa su color, y pasa al siguiente».

Ejemplo unidimensional: lectura de un array

 Copia el siguiente programa y pégalo en la Playgroud.

Fichero: **for_en_array.rs**

```
fn main() {
    let numeros = [2, 4, 34, 98];

    // numeros.len() es 4, así que el rango va de 0 a 3
    for i in 0..numeros.len() {
        println!("Índice: {}, Valor: {}", i, numeros[i]);
    }
}
```

- El array **numeros** definido tiene cuatro elementos, por tanto **numeros.len()** es igual a 4. Es la longitud del array.
- El rango **0..numeros.len()** genera los números 0, 1, 2, 3
- En el bucle **for**, la **i** empezará por tomar el valor 0 en la primera vuelta, 1 en la segunda vuelta, 2 en la tercera vuelta y 3 en la cuarta vuelta.

Por tanto, al ejecutar este programa obtendremos:

```
Índice: 0, Valor: 2
Índice: 1, Valor: 4
Índice: 2, Valor: 34
Índice: 3, Valor: 98
```

Pulsa el botón **[RUN]** de tu Playground y comprueba el resultado.

 **Nota:** Para interpretar los resultados sustituye el valor de **i** en cada pasada del bucle.

Ejemplo bidimensional: lectura de una matriz

 Copia el siguiente programa y pégalo en la Playgroud.

Fichero: **for_en_matriz.rs**

```
fn main() {
    // Matriz de 2 filas (alto) y 3 columnas (ancho)
    let matriz = [
        [2, 9, 32],
        [65, 90, 4]
    ];

    let alto = 2;
    let ancho = 3;

    // Bucle externo para recorrer las filas (2)
    for fila in 0..alto {
        // Bucle interno para recorrer las columnas (3)
        for columna in 0..ancho {
            print!("{}", matriz[fila][columna]);
        }
        println(); // Salto de línea al terminar cada fila
    }
}
```

Es similar al ejercicio anterior pero ahora tenemos un bucle externo que recorre las filas (fila toma los valores 0 y 1) y un bucle interno que para cada fila, recorre las columnas (columna toma los valores 0, 1 y 2).

De esta forma se recorren y se leen los seis elementos de la matriz.

Resultado de ejecutar el programa

```
2 9 32
65 90 4
```

5.5. Sección 5: Bucles infinitos loop y while: repetir tareas sin cansarse

En esta sección veremos cómo podemos repetir un conjunto de instrucciones vinculado al cumplimiento o no de una condición.

5.5.1. El bucle loop

A los ordenadores se les da fatal improvisar, pero son los reyes absolutos de la repetición. No se cansan, no se aburren y no cometen errores por falta de atención. En Rust, cuando queremos que el ordenador repita un bloque de código, usamos los **bucles** (loops). El primero y más directo se llama, literalmente, **loop**.

 Copia este código en tu Playground y prepárate para usar el botón de emergencia

Fichero: **bucle_loop.rs**

```
fn main() {
    let mut contador = 0;

    println!("{}", "Iniciando el motor de repetición!");
```

```
// La palabra 'loop' abre un bucle infinito.
// Lo que esté dentro del loop se repetirá de principio a fin
// para siempre...
loop {
    contador = contador + 1;
    println!("Vuelta número: {}", contador);

    // ✗ Si dejamos el código así, la Playground se colgará.
    // Necesitamos una condición de salida (un freno de mano).
    // Es el contador de la línea de abajo.
    if contador == 5 {
        break; // 💡 ¡La palabra 'break' rompe el bucle, nos saca de
aquí
// y nos lleva a la siguiente instrucción después del
loop
    }
}

println!("¡Bucle terminado con éxito! El contador final es: {}", contador);
}
```

⚙️ Experimento 1: El freno de mano (break)

Si pulsas **[RUN]**, verás que el programa cuenta del 1 al 5 a la velocidad del rayo y se detiene limpiamente. La palabra clave `break` (que en inglés significa romper o frenar) es nuestra válvula de escape.

- **Misión (Simular un desastre controlado):** Vamos a ver qué pasa si sabotear el freno de mano. Aplica el control de daños al revés: añade dos barras `//` al principio de la línea que tiene el `break`; Vuelve a pulsar **[RUN]**.
- **¿Qué pasa?** Verás que los números empiezan a subir sin parar en la pantalla. ¡El programa ha entrado en un bucle infinito! Pulsa la pestaña **[Close]** en la parte superior derecha de la Playground para rescatar al ordenador.
- **Corrección:** Quita las barras `//` para devolver el `break`; a su sitio y que el programa vuelva a ser seguro.

🧠 La lección clave: Las tres piezas del bucle loop

Para que un bucle `loop` no se vuelva infinito, siempre necesitas tres cosas:

1. Una variable mutable fuera del bucle (**`let mut contador = 0;`**) para llevar la cuenta.
2. Una actualización dentro del bucle (**`contador = contador + 1;`**) para que las cosas avancen.
3. Un interruptor de apagado (**`if ... break;`**) para saber cuándo hemos terminado el trabajo.

5.5.2. El Bucle while

El bucle `loop` que vimos antes es muy potente, pero tener que escribir un **`if`** y un **`break`** dentro del bucle cada vez, puede ser un poco pesado. Por suerte, Rust tiene un hermano más inteligente llamado **`while`** (que en inglés significa «mientras»).

El bucle **while** funciona mediante una condición: repite el código en su interior mientras esa condición —que se evalúa en cada pasada del bucle— sea verdad. En el momento en que deja de serlo, el bucle se frena solo automáticamente y el programa sigue en la siguiente instrucción al **while**.

 Copia este código en tu Playground para ver cómo funciona un medidor de batería que se va descargando

Fichero: **bucle_while.rs**

```
fn main() {
    // Empezamos con la batería al 100%
    let mut bateria = 100;

    println!("📱 Teléfono encendido. Batería al {}", bateria);

    // MIENTRAS la batería sea mayor que cero, el teléfono sigue funcionando:
    while bateria > 0 {
        // En cada vuelta del bucle, el teléfono gasta un 25% de energía
        bateria = bateria - 25;

        println!("Usa una aplicación... Batería restante: {}", bateria);
    }

    // ❌ Error oculto en tu cuaderno:
    // ¿Qué pasaría si dentro del bucle olvidamos restar energía?
    // (Ejemplo: // bateria = bateria - 25;)

    println!("🔋 ¡Batería agotada! El teléfono se ha apagado.");
}
```

⚙️ Experimento 1: La descarga controlada

Si pulsas **[RUN]**, verás cómo el programa va calculando la bajada de batería limpiamente (100% -> 75% -> 50% -> 25% -> 0%) y, al llegar a cero, la condición `bateria > 0` deja de cumplirse. Rust lo detecta, sale del bucle y te muestra el mensaje final de «🔋 ¡Batería agotada! El teléfono se ha apagado.». ¡Todo automático!

⚙️ Experimento 2: El tiempo congelado

Vamos a provocar un fallo de lógica muy común entre programadores novatos.

- **Misión:** Comenta la línea donde restamos la energía añadiendo dos barras al principio: `// bateria = bateria - 25;`. Vuelve a pulsar **[RUN]**.
- **¿Qué ocurre?** Como la batería nunca baja de 100, la condición `bateria > 0` siempre es verdadera. El programa se queda atrapado en un bucle infinito mostrando el mismo mensaje en bucle. Pulsa sobre la pestaña **[Close]** en la parte superior derecha de la Playground para rescatar al ordenador.
- **Corrección:** Borra las barras `//` para que el teléfono vuelva a gastar energía de forma normal.

🧠 La diferencia fundamental entre `loop` y `while`

- Usa loop cuando no sepas de antemano cuántas vueltas vas a dar y necesites comprobar cosas complejas dentro.
- Usa while cuando tengas una condición matemática clara y directa (como un contador o un porcentaje) para controlar la duración del bucle.

5.6. Sección 5: Proyectos clave

Llegados a este punto, ya no eres un mero espectador; tienes en tu caja de herramientas los pilares de la programación en Rust: variables, inmutabilidad, tipos de datos, decisiones y bucles. Es hora de unir todas las piezas en proyectos reales.

5.6.1. Conversor de unidades interactivo

¿Cuántas veces has tenido que buscar en **Google** cuántos kilómetros son tantas millas, o a cuántos grados **Celsius** equivale una temperatura en **Fahrenheit**? En este proyecto vamos a programar nuestro propio conversor automático.

 Copia este código en tu Playground

Fichero: **conversor_de_unidades.rs**

```
fn main() {
    // === CONFIGURACIÓN DEL USUARIO (Cambia estos valores para probar) ===
    let millas_a_convertir = 5.0;    // Usa números con punto decimal (f64)
    let celsius_a_convertir = 25.0;
    // =====

    println!("⚙️ INICIANDO CONVERSOR DE UNIDADES INTERACTIVO ⚙️\n");

    // 1. Conversión de Distancia (Millas a Kilómetros)
    // Regla matemática: 1 milla = 1.60934 kilómetros
    let factor_millas = 1.60934;
    let kilometros_resultantes = millas_a_convertir * factor_millas;

    println!("📍 [DISTANCIA]: {} millas equivalen a {:.2} kilómetros.",
             millas_a_convertir, kilometros_resultantes);

    // 2. Conversión de Temperatura (Celsius a Fahrenheit)
    // Regla matemática: F = (C * 9/5) + 32
    let fahrenheit_resultantes = (celsius_a_convertir * 9.0 / 5.0) + 32.0;

    println!("🌡️ [TEMPERATURA]: {}°C equivalen a {:.1}°F.",
             celsius_a_convertir, fahrenheit_resultantes);

    // ❌ Error de laboratorio provocado:
    // ¿Qué pasaría si intentas sumar un número entero a uno decimal?
    // Descomenta la línea de abajo quitando las barras para ver el enfado de
    Rust:
    // let error_calculo = millas_a_convertir + 10;
}
```

⚙️ Experimento 1: Rompiendo las matemáticas de Rust

Si pulsas **[RUN]**, el programa calculará perfectamente las conversiones y te mostrará los resultados limpios en la pantalla derecha gracias a un truco visual: **{:.2}** le dice a Rust que solo muestre dos decimales para que no se llene la pantalla con demasiados números.

Ahora, hagamos una prueba de fuego con la seguridad de Rust. Quita las dos barras `//` de la última línea (`let error_calculo = ...`) y pulsa **[RUN]**.

- **El resultado:** Rust se detendrá con un error de tipo. No te permite sumar **millas_a_convertir** (que es un número decimal, `f64`) con el número **10** (que es un número entero, `i32`).
- **La solución:** Rust te obliga a ser ultra-preciso. Si quieres sumar diez a **millas_a_convertir**, tendrías que escribir **10.0**. Vuelve a colocar las barras `//` para dejar el laboratorio limpio

 ¡Te toca jugar!

Modifica los valores de las líneas 3 y 4. Pon las millas que recorres para ir al colegio o la temperatura actual de tu habitación. Pulsa **[RUN]** y observa cómo tu código se adapta al instante a tus nuevos datos.

5.6.2. Geneador de historias aleatorias (tipo Mad Libs)

¿Has jugado alguna vez a **Mad Libs**? Es ese juego divertidísimo en el que le pides a un amigo que te diga un nombre, un verbo, un adjetivo y un lugar sin saber la historia, y al rellenar los huecos queda un relato completamente absurdo y desternillante.

En este último proyecto del laboratorio, vamos a construir un Generador de Historias Locas. Tu ordenador se encargará de fusionar las palabras que tú elijas dentro de un texto fijo.

 Copia este código en tu Playground

Fichero: **generador_historias_aleatorias.rs**

```
fn main() {
    // === CONFIGURACIÓN DE TU HISTORIA (¡Cambia estas palabras!) ===
    let protagonista = "Un robot con sombrero";
    let lugar = "la cocina de la abuela";
    let objeto = "una cuchara de madera";
    let accion = "bailar la macarena";
    let año = 2085; // Un número entero
    // =====

    println!("📖 GENERADOR DE HISTORIAS MÁGICAS ACTIVADO 📖\n");

    // Rust se encarga de tejer la historia usando los marcadores {}
    println!("Corría el año {}, cuando ocurrió algo inaudito.", año);
    println!("El valiente {} se encontraba en medio de {}. ", protagonista, lugar);

    println!("De repente, sacó {} de su mochila y, sin pensarlo dos veces,", objeto);
    println!("¡se puso a {} frente a todos los presentes!", accion);

    println!("\n🎉 Fin de la aventura. ¡Vuelve a cambiar las variables para
```

```
otra historia!");  
}
```

❗ **Nota:** Cuando se imprime la secuencia de caracteres `\n` con la instrucción `println!`, equivale a hacer un salto de línea y retorno de carro.

⚙️ Experimento 1: El tejedor de palabras

Si pulsas **[RUN]**, verás cómo Rust recoge ordenadamente el texto de cada una de tus variables y lo encaja exactamente en el orden en que has colocado los marcadores `{}` dentro de los `println!`.

Fíjate en un detalle: hemos mezclado variables de texto (`&str`) con una variable de **número entero** (año). A diferencia de las matemáticas (donde vimos que no se pueden mezclar tipos), a la hora de imprimir en pantalla, Rust es muy amable y te permite colocar cualquier tipo de dato dentro de las llaves `{}` porque sabe convertirlo todo a texto visual.

🎮 ¡Hazlo tuyo!

Cambia por completo las palabras de las líneas 3 a 7. Inventa el protagonista más raro que se te ocurra (un pingüino astronauta, tu profesor de matemáticas, un zombi vegetariano...), un lugar extraño y una acción ridícula. Modifica también el año. Vuelve a pulsar **[RUN]** y observa cómo tu programa genera una historia completamente nueva en un abrir y cerrar de ojos.

6. 🚀 Tipos de datos y operadores en Rust

En este tema hablaremos con más rigor sobre los tipos de datos en Rust e introduciremos los operadores en Rust. Un operador es un símbolo que actúa normalmente sobre dos valores de entrada y produce un valor de salida.

6.1. Tipos de datos en Rust

Hasta ahora, hemos visto que nuestros programas manejan tres tipos de información: Números, Texto y Booleanos (sí/no). En el mundo real esto es un poco más complejo, pero no quisimos mostrártelo antes para no complicarte la vida. ¡Queríamos que empezaras a jugar con los programas lo antes posible!

Ya conoces algunos tipos como **i32** (enteros) o **f64** (decimales). Un entero i32 ocupa siempre en la memoria 32 bits y un decimal f64 ocupa siempre 64 bits. Veremos a continuación el concepto de bit. También has visto el texto en formato **&str**. Este último es una versión **recortada** de otro tipo llamado **String**. Ambos guardan texto, pero lo hacen de formas distintas y su tamaño en la memoria cambia según lo largo que sea el mensaje a diferencia de los anteriores que son de tamaño fijo.

Para entender por qué Rust es tan estricto con los tipos de datos, tenemos que hacer un viaje relámpago al interior de la memoria de tu ordenador.

6.1.1. 🟢 ¿Qué es un bit?

Un bit es la **célula más elemental** que existe en la informática. Imagínalo como un interruptor de la luz diminuto: solo puede estar encendido (1) o apagado (0).

Como un solo interruptor no sirve para guardar mucha información, los ordenadores agrupan estos bits en **paquetes** de 8, llamados **Bytes**. Una buena aproximación es imaginarse la memoria RAM de un ordenador como una sucesión de estos Bytes.

Así tendremos que:

- Un número de tipo **i32** se guarda en la memoria usando 4 bytes (32 bits en total).
- Un número de tipo **f64** se guarda en la memoria usando 8 bytes (64 bits en total).

6.1.2. 🌐 La memoria del ordenador y el sistema binario

El sistema binario (combinación de unos y ceros) puede representar cualquier cosa: números positivos, negativos, decimales y letras. Pero aquí está el gran secreto: la memoria del ordenador es ciega. Solo ve una fila gigantesca de unos y ceros y tenemos que decirle cómo está almacenada la información, con qué código. Cual es la sucesión de tipos de datos en esa fila. Cuando especificamos **i32** o **f64** le estamos proporcionando ese código.

Imagina que guardas un número f64. El ordenador mete en su memoria una secuencia de 64 interruptores encendidos y apagados. Para poder recuperar ese número más tarde, Rust necesita saber dos cosas obligatoriamente:

1. ¿En qué casilla de la memoria empieza el número?
2. ¿Qué tipo de dato es? (Es decir, qué «traductor» debe usar cuando lee el número).

Si intentaras leer esos 64 bits como si fuera un número entero `i32`, el ordenador solo leería la mitad del número (32 bits) y la traducción sería un desastre total. Es como si intentaras leer un libro en español usando las reglas de pronunciación del alemán: las letras son las mismas, ¡pero no entenderías nada!

Además, el ordenador usa trucos diferentes (códigos) para guardar números negativos o para convertir letras en números (asignando a cada letra un código numérico).

Por eso, los tipos de datos son las etiquetas que le dicen a Rust cómo traducir los unos y ceros de la memoria para que tu programa los entienda y no se vuelva loco.

6.1.3. 🤖 Tipos de datos por defecto

Cuando asignamos un dato a una variable —en este caso el dato se llama un literal—, aunque no indiquemos el tipo de dato Rust asigna a la variable un tipo que denominamos el tipo por defecto.

Números decimales

En los ejemplos hemos utilizado instrucciones como esta:

```
let v = 50.5;
```

Aquí definimos la variable inmutable **v** (con **let**) como un valor decimal pero no indicamos el tipo de dato de **v**. En Rust existen dos tipos de datos decimales, `f32` y `f64`. Si no indicamos el tipo en la asignación de un valor decimal a una variable como en la anterior instrucción, el compilador toma por defecto **f64**. Por tanto **v** será del tipo `f64`.

Si queremos realmente que **v** sea de tipo `f32` lo tenemos que indicar en la declaración o en la asignación. Por ejemplo:

```
let v:f32 = 50.5;
```

Arriba indicamos que **v** es de tipo `f32`.

Una forma alternativa de expresar lo mismo es:

```
let v = 50.5f32;
```

Aquí estamos diciendo que la constante 50.5 hay que tomarla como `f32` por lo que **v** tendrá el mismo tipo `f32`. A esto se le llama inferencia de tipos, cuando el compilador adivina el tipo de la variable aún sin haber indicado el tipo en la misma variable.

Números enteros

Con los enteros `i32` e `i64` ocurre algo similar. El tipo por defecto es `i32` pero también existe `i64`. Entonces:

```
let edad = 20;           // edad es i32. Por defecto, sin indicar nada
let edad: i32 = 20;      // edad es i32. Reiterativo
```

```
let edad = 20i64; // edad es i64
let edad:i64 = 20; // edad es i64
```

Los tipos de datos numéricos que definiremos a continuación tienen tipos por defecto.

6.1.4. 🧱 Las dos familias de tipos en Rust

Para organizar todos los tipos de datos, Rust los divide en dos grupos muy fáciles de entender: los **Escalares** y los **Compuestos**.

6.1.4.1. Tipos Escalares (Un solo valor)

Son los tipos de datos más sencillos. Representan **un único valor atómico** (no se pueden dividir en partes más pequeñas). Imagínalos como piezas individuales de Lego.

- **Enteros (Números sin decimales):** Sirven para contar cosas enteras (como vidas en un videojuego o años). Rust te permite elegir el tamaño exacto que ocuparán en memoria:
 - Con signo (pueden ser negativos o positivos, "i" de integer): i8, i16, i32 (el favorito por defecto), i64, i128.
 - Sin signo (solo positivos o cero, "u" de unsigned): u8 (ideal para colores en informática, de 0 a 255), u16, u32, u64, u128.
 - Según el ordenador: isize y usize (su tamaño depende de si el procesador es de 32 o 64 bits).
- **Flotantes (Números con decimales):** Para cuando necesitas precisión (como la salud de un jefe con 99.5% de vida o el precio de un juego).
 - f32 (precisión simple).
 - f64 (precisión doble, el que Rust usa por defecto).
- **Booleanos (Verdad o mentira):** Solo tienen dos estados posibles. ¡Como un interruptor!
 - **bool:** Sus únicos valores son true (verdadero) o false (falso). Es el motor de las decisiones en el código cuando se utiliza (if).
- **Caracteres (Una sola letra o símbolo):** Sirven para guardar una única letra, número suelto o... ¡incluso un emoji!
 - **char:** Se escriben con comillas simples (por ejemplo: 'A', '7' o '🚀'). Ocupan 4 bytes porque usan Unicode, lo que les permite representar casi cualquier símbolo del mundo.

6.1.4.2. Tipos Compuestos (Varios valores juntos)

Son tipos que permiten agrupar múltiples valores dentro de una sola **caja** o **variable**. Nos podremos referir a esos múltiples valores a través del nombre único de la variable.

- **Tuplas (Tuples):** Son como una mochila donde puedes meter cosas de diferentes tipos, pero con un tamaño fijo: una vez que creas la tupla, no puede crecer ni encoger.
 - Ejemplo: Puedes guardar el nombre de un jugador (&str), su puntuación (i32) y si está vivo (bool) en una sola variable:

```
let jugador = ("Halcón", 2500, true);
```

Se caracterizan por utilizar un paréntesis en su declaración y los distintos valores separados por comas.

- **Arrays / Arreglos (Matrices fijas):** Son como un cartón de huevos. Guardan una lista de elementos, pero con dos reglas estrictas: todos tienen que ser del mismo tipo y el tamaño una vez creado el array es fijo (no puede cambiar).
 - Ejemplo: Las notas de 5 exámenes de un alumno: [10, 8, 9, 7, 9]. Se caracterizan por utilizar corchetes en su declaración con la lista de elementos separados por comas.

```
let notas = [10, 8, 9, 7, 9];
```

Piensa un poco en esto: Si tuvieras que guardar el inventario de Minecraft de un jugador, ¿qué tipo usarías para la cantidad de bloques de piedra? ¿Y para el nombre del pico?

6.1.5. Ejemplo que utiliza diferentes tipos

Después del listado tienes explicaciones para entender el programa.

 Copia este código en tu Playground

Fichero: **ejemplos_tipos.rs**

```
#![allow(warnings)] // <-- ¡Esta línea mágica:elimina todos los avisos
amarillos en este fichero!

fn main() {
    // --- 1. TIPOS ESCALARES (Un solo valor) ---

    // Enteros: Rust adivina que es i32 por defecto
    let vidas = 3;
    // Obligamos a que sea un u8 (solo de 0 a 255), ideal para ahorrar memoria
    let nivel: u8 = 10;

    // Flotantes: Números con decimales
    let puntuacion = 95.5; // f64 por defecto
    let gravedad: f32 = 9.81; // Forzamos precisión simple f32

    // Booleanos: ¡Verdadero o Falso!
    let partida_terminada = false;
    let tiene_llave = true;

    // Caracteres: ¡Ojo! Van con comillas SIMPLES ''
    let inicial = 'H';
    let mi_emoji = '🚀'; // ¡Sí, Rust acepta emojis en los caracteres!

    // --- 2. TIPOS COMPUESTOS (Varios valores juntos) ---

    // Tupla: Un combo de datos de diferentes tipos (Nombre, Puntuación, ¿Está vivo?)
    let jugador: (&str, i32, bool) = ("Halcón", 2500, true);

    // Para sacar los datos de la tupla usamos un punto y su posición
    (empezando desde 0)
```

```

let nombre_jugador = jugador.0;
let puntos_jugador = jugador.1;
let jugador_esta_vivo = jugador.2;

// Array: Una lista fija de elementos que TIENEN que ser del mismo tipo
// Guardamos las puntuaciones de las últimas 4 partidas
let historial_puntos: [i32; 4] = [120, 98, 101, 63]; // 4 valores de tipo
i32

// Para sacar un dato del array usamos corchetes [] y la posición (el
primero es el 0)
let primera_partida = historial_puntos[0];

// --- ¡VAMOS A MOSTRARLO EN PANTALLA! ---
println!("¡Bienvenido al juego, {}!", nombre_jugador);
println!("Tu inicial es la {} y tienes {} vidas.", inicial, vidas);
println!("En tu primera partida ganaste {} puntos y ahora tienes {}.\"",
primera_partida, puntos_jugador);
println!("¿Estás listo para despegar? {}", mi_emoji);
}

```

Las siguientes explicaciones están encaminadas a que entiendas el programa anterior. Ejecuta el programa y compara los resultados obtenidos con los que puedes deducir tu mismo de las explicaciones. Si no llegas a comprender el 100% del programa no es problemático pero si haces un esfuerzo te será útil.

🎨 Rust empieza a contar desde 0

En todos los tipos de datos compuestos donde bajo el nombre único de una variable se agrupan varios valores, se utiliza siempre un **índice** que empieza por cero para el primer valor y luego se va aumentando de uno en uno. La forma de utilizar ese índice es diferente en las tuplas **nombre_variable.0**, **nombre_variable.1**, etc. y en los vectores **nombre_variable[0]**, **nombre_variable[1]**, etc.

Lo podemos ver en los siguientes ejemplos:

En la tupla

```

let jugador: (&str, i32, bool) = ("Halcón", 2500, true);

// Para sacar los datos de la tupla usamos un punto y su posición
// (empezando desde 0)
let nombre_jugador = jugador.0;
let puntos_jugador = jugador.1;
let jugador_esta_vivo = jugador.2;

```

En el vector

```

let historial_puntos: [i32; 4] = [120, 98, 101, 63];

// Para sacar un dato del array usamos corchetes [] y la posición

```

```
// (el primero es el 0)
let primera_partida = historial_puntos[0];
let segunda_partida = historial_puntos[1];
// etc.
```

🍷 Comillas simples o dobles

Las comillas importan: Las letras sueltas (**char**) llevan comillas simples ('A'), mientras que los textos de más de una letra (**&str** o **String**) llevan comillas dobles ("Halcón"). Si mezclas las comillas, ¡el compilador te echará una bronca de campeonato!

6.1.6. Gestión de Texto: La Diferencia entre &str y String

En Rust, el texto (las cadenas de caracteres) no se maneja de una sola forma. El lenguaje divide el texto en dos tipos principales según dónde se guarda la información y si se puede modificar o no. Estos dos tipos son **String** y **&str**.

Para entenderlo de forma sencilla:

- **&str (Cita de texto / String Slice)**: Es como un cartel de «No tocar». Es texto de longitud fija que solo sirve para ser leído.
- **String (Texto dinámico)**: Es como una libreta donde puedes escribir, borrar y añadir más páginas. Es texto modificable que puede crecer o encogerse.

Tabla Comparativa Rápida

Características de una variable de tipo &str o String

Característica	&str	String
¿Se puede modificar?	No (Inmutable)	Sí (Mutable)
¿Dónde se guarda?	En la memoria fija (Binario)	En el montón (Heap)
¿Tiene dueño?	Es prestado (Referencia)	Es propietario del texto
Uso principal	Lectura rápida y fija	Crear y transformar texto

6.1.6.1. El tipo &str (Referencia de cadena)

Es el tipo por defecto cuando escribes texto directamente entre comillas en tu código. Es muy rápido y eficiente porque el ordenador sabe exactamente cuánto mide desde el principio y que no va a cambiar.

```
fn main() {
    // Esto es un &str por defecto
    let saludo = "¡Hola, mundo!";

    // NO puedes añadirle caracteres (daría error):
    // saludo.push_str(" ¿Cómo estás?");
}
```

6.1.6.2. 2. El tipo String

Este tipo se utiliza cuando no sabes qué va a escribir el usuario, o cuando necesitas pegar varios textos juntos para formar una frase dinámica (sobre la marcha).

```
fn main() {
    // Creamos un String a partir de un texto fijo
    let mut mensaje = String::from("Hola");

    // ¡Sí podemos modificarlo y añadirle más texto!
    mensaje.push_str(" alumno de Rust");

    println!("{}", mensaje); // Imprime: Hola alumno de Rust
}
```

6.2. Los motores del cambio: Operadores en Rust

Imagina que las variables son cajas donde guardas cosas (como tus monedas, tu nivel o tu inventario). Los operadores son las herramientas matemáticas y lógicas que te permiten hacer cosas con esas cajas: sumar puntos, comparar quién ha ganado o comprobar si tienes la llave correcta para abrir una puerta.

En Rust tenemos tres familias principales de operadores que vas a usar todo el tiempo.

6.2.1. 🇺🇸 Operadores Matemáticos (Los de toda la vida)

Sirven para hacer cálculos. Funcionan exactamente igual que en tu clase de matemáticas, con un par de símbolos especiales que usa el ordenador.

Operador	Operación	Ejemplo en Rust	Resultado
+	Suma	let total = 5 + 3;	8
-	Resta	let vidas = 10 - 2;	8
*	Multiplicación	let doble = 4 * 2;	8
/	División	let mitad = 16 / 2;	8
%	Módulo (El resto de la división)	let resto = 9 % 2;	1 (9 entre 2 da 4, y sobra 1)

- ⚠️ **La regla de oro de Rust:** Rust es superestricto. No puedes mezclar tipos de números en las operaciones. Si intentas sumar un entero (i32) con un decimal (f64), el compilador se enfadará y detendrá el programa. ¡Tienen que ser del mismo tipo!

6.2.2. ⚖️ Operadores de Comparación (Los jueces)

Sirven para comparar dos cosas. El resultado de estas operaciones siempre es un booleano (true o false). Son los que usarás para que tu código tome decisiones (por ejemplo: «Si las vidas son iguales a 0, entonces *Game Over*»).

Operador	Significado	Ejemplo	Qué responde Rust?
==	¿Es exactamente igual? (¡Ojo, doble signo igual!)	5 == 5	true
!=	¿Es diferente?	5 != 3	true
>	¿Es mayor que?	4 > 7	false
<	¿Es menor que?	3 < 8	true

>=	¿Es mayor o igual que?	5 >= 5	true
<=	¿Es menor o igual que?	10 <= 2	false

💡 **Truco mental:** No confundas = con ==. Un solo = sirve para guardar algo en una caja (let x = 5;). El doble == sirve para preguntar si dos cajas tienen lo mismo.

6.2.3. 🧠 Operadores Lógicos (Los detectives)

Sirven para combinar varias preguntas a la vez. Imagina que para entrar a una mazmorra necesitas: tener el nivel 10 Y tener la llave dorada. ¡Aquí entran en juego los operadores lógicos!

- **&& (Operador Y / AND):** Da true solo si todas las condiciones son verdaderas.
 - let **entrar** = (nivel >= 10) && tiene_llave; (Si una de las dos es false, **entrar** es false).
- **|| (Operador O / OR):** Da true si al menos una de las condiciones es verdadera. (El signo | se obtiene con la tecla **Alt Gr + 1** en la mayoría de teclados).
 - let jugar = (tengo_consola) || (tengo_pc); (Con que tengas uno de los dos, ya puedes jugar).
- **! (Operador NO / NOT):** Invierte el resultado. Lo que era true lo vuelve false, y viceversa. Es el botón de «llevar la contraria».
 - let vivo = true;
 - let fantasma = !vivo; (fantasma pasará a valer false).

💻 Un minireto

¿Qué valor crees que guardará la variable resultado_final en este código?

```
let puntos = 100;
let tiene_bonus = true;
let resultado_final = (puntos > 50) && (tiene_bonus == true);
```

6.2.4. 🎮 Ejercicio propuesto: Misión «Acceso a la Mazmorra del Dragón»

¡Ha llegado el momento de poner a prueba tus nuevos superpoderes con los operadores! Imagina que estás programando las reglas de un juego de rol. El jugador quiere abrir la puerta de una mazmorra secreta, pero el juego solo le dejará pasar si cumple unos requisitos muy estrictos.

📝 El Reto

Tienes que escribir un programa en tu Playground que calcule la puntuación final de un jugador y decida si puede entrar a la mazmorra.

Las reglas del juego son:

- La **Puntuación Total**: Se calcula sumando los **puntos_base** más los **puntos_extra** multiplicados por 2.
- La **Condición de Entrada**: Para entrar, el jugador necesita tener una **puntuacion_total** **mayor o igual a 80** Y poseer el objeto mágico (**tiene_llave**).

Si todavía no has llegado a una solución...

 Copia este código en tu Playground, comprende las operaciones y la lógica y ejecútalo. Cambia algunos valores y condiciones a tu gusto y observa los resultados al ejecutarlo

 Fichero: **operadores_reto.rs**

```
#![allow(warnings)] // Evitamos los avisos amarillos para concentrarnos

fn main() {
    // --- DATOS DEL JUGADOR (Puedes cambiar estos valores para hacer pruebas)
    ---
    let puntos_base = 40;
    let puntos_extra = 25;
    let tiene_llave = true;

    // --- 1. OPERACIÓN MATEMÁTICA ---
    // Multiplica los puntos_extra por 2 y súmaselos a los puntos_base
    let puntuacion_total = puntos_base + (puntos_extra * 2);

    // --- 2. OPERADOR DE COMPARACIÓN Y LÓGICO ---
    // Comprueba si la puntuación es mayor o igual a 80 Y ADEMÁS tiene la llave
    let puede_entrar = (puntuacion_total >= 80) && (tiene_llave == true);

    // --- 3. EL VEREDICTO DEL ORDENADOR ---
    println!("--- ESTADO DE LA MISIÓN ---");
    println!("Puntuación final conseguida: {} puntos.", puntuacion_total);
    println!("¿Tiene el objeto mágico?: {}", tiene_llave);
    println!("¿Se abre la puerta de la mazmorra?: {}", puede_entrar);
}
```

7. Subiendo de nivel: Instalación de un Entorno de Desarrollo Profesional

Objetivo: Trabajo con imágenes

Si has llegado hasta aquí, ya has jugado con Rust en la Playground, has corregido errores en rojo y has creado tus primeros minijuegos.

¡Felicidades! Pero la Playground tiene una limitación: vive en Internet y no puede tocar los archivos de tu ordenador.

¿Te imaginas programar un código en Rust que abra una foto tuya, la vuelva en blanco y negro, la pixelee o le dibuje figuras encima? Para trabajar con tratamiento de imágenes, necesitamos dar el salto al mundo profesional.

Vamos a instalar el motor de Rust directamente en tu máquina.

7.1. Instalación en LINUX

Si usas **Linux**, juegas con ventaja: Rust se siente en casa en este sistema operativo y la instalación es limpiísima.

- **Distribuciones soportadas (Lista veraz):**
 - Familia Debian: Ubuntu (versión 20.04 en adelante), Linux Mint, Pop!_OS, Zorin OS, Kubuntu, Xubuntu.
 - Familia Red Hat: Fedora, CentOS.
 - Familia Arch: Arch Linux, Manjaro.
- **Lista de programas a instalar:**
 - **Herramientas de construcción** (build-essential): Rust necesita un **enlazador** (linker) para unir el código compilado.
 - **curl**: Programa auxiliar para instalar Rust.
 - **Rustup & Cargo**: El motor oficial y el gestor de paquetes de Rust.
 - **Visual Studio Code (VS Code)**: Nuestro editor de código profesional.

Instrucciones de instalación paso a paso en Linux:

Paso 1: Instalar los cimientos. Abre tu terminal y escribe el comando para instalar las herramientas base (en Ubuntu/Debian):

```
sudo apt update && sudo apt install build-essential -y
```

Paso 2: Instalar curl. Abre tu terminal y ejecuta el siguiente comando:

```
sudo apt install curl -y
```

Paso 3: Instalar Rust. Copia y pega este comando oficial en la terminal para descargar e instalar Rust automáticamente:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Cuando te pregunte en la pantalla, pulsa la tecla 1 y luego ENTER para elegir la instalación por defecto.

Paso 4: Activar el motor. Cierra la terminal y vuélvela a abrir, o escribe este comando para que tu sistema sepa dónde está Rust:

```
source "$HOME/.cargo/env"
```

Paso 5: El editor visual. Instala Visual Studio Code desde la tienda de software de tu distribución o desde su web oficial.

7.2. 🖥️ Instalación en WINDOWS

Windows requiere un par de pasos extra porque, por defecto, no viene preparado de fábrica para compilar lenguajes de alto rendimiento como Rust.

- Versiones soportadas: Windows 10 y Windows 11 (de 64 bits).
- Lista de programas a instalar:
 - **Herramientas de C++ de Visual Studio (MSVC):** Los cimientos que exige Rust en Windows. Hay que instalarlas en primer lugar.
 - **Rustup & Cargo** (Desde el instalador oficial de Rust para Windows).
 - **Visual Studio Code.** (Nuestro editor profesional)

🚀 Instrucciones de instalación paso a paso en Windows:

- **Paso 1: Los cimientos (¡Crucial!).** Para que Rust funcione en Windows, necesita las **herramientas de compilación de Microsoft C++**. No te preocupes, no hace falta que instales todo el programa pesado de Visual Studio, solo utilizaremos una versión ligera.
 1. Entra en la página oficial de [Descargas de Visual Studio](#).
 2. No le des al botón de descargar la versión Community que sale arriba del todo.
 3. Baja por la página web hasta encontrar una sección desplegable llamada **Herramientas para Visual Studio** (o **Tools for Visual Studio**).
 4. Desplégala y encontrarás una subsección titulada **Herramientas de compilación para Visual Studio 2026** con un botón a la derecha para descargar. Haz clic en **Descargar**.
 5. Al abrir el archivo descargado se iniciará el asistente. En la ventana de selección de componentes, asegúrate de marcar la casilla **Desarrollo para el escritorio con C++** en la esquina superior izquierda.
 6. Pulsa en Instalar. (Aviso: tardará unos minutos porque, aunque es una versión reducida, sigue pesando bastante. ¡Paciencia!)
- **Paso 2: El instalador de Rust.** Ve a la web oficial rustup.rs, pulsa sobre el enlace **display all supported installers**, y descarga el archivo ejecutable **rustup-init.exe**. Haz doble clic en él. Se abrirá una pantalla negra de consola. Pulsa la tecla 1 y luego ENTER. ¡Rust ya está en tu sistema!

Para que se reconozca la instalación, cierra la terminal que has estado utilizando y vuelve a abrir otra terminal.

- **Paso 3: El editor visual.** Descarga e instala **Visual Studio Code** desde su [página oficial](#).

7.3. 🚀 Instrucciones en macOS

- **Paso 1: Activar las herramientas de Apple.** Abre la aplicación **Terminal** (búscala en tu Spotlight con **Cmd + Espacio**) y escribe este comando:

```
xcode-select --install
```

Si no están instaladas, te aparecerá una ventana emergente en tu Mac. Dale a **Instalar** y acepta los términos.

Si ya están instaladas no tienes que hacer nada.

- **Paso 2: Descargar Rust.** En esa misma terminal, pega el comando universal de instalación:

```
curl --proto 'https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Pulsa 1 y luego ENTER cuando te lo pida la pantalla.

- **Paso 3: El editor visual.** Descarga Visual Studio Code para Mac desde la web oficial, arrástralo a tu carpeta de Aplicaciones y ábrelo.

7.4. 🛠️ Configuración de Visual Studio Code (Para todos los sistemas)

Una vez instalado VS Code en tu ordenador, tenemos que tunearlo para que entienda Rust a la perfección. Abre VS Code, haz clic en el icono de Extensiones en la barra lateral izquierda (parecen 4 cuadraditos flotantes como se muestra abajo en la segunda imagen) e instala las tres extensiones que te indicamos después de leer estas instrucciones:

Fíjate en estos dos botones en la barra lateral izquierda de VS Code:



Figura 2: Botón de ficheros. Para visualizar los ficheros de nuestro directorio (cuando lo tengamos) en el panel izquierdo de VS Code.

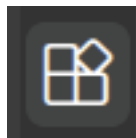


Figura 3: Botón de extensiones. Para visualizar las extensiones y el cuadro de búsqueda en el panel izquierdo.

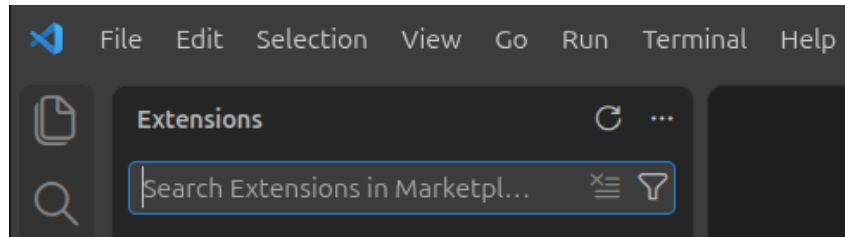


Figura 4: Cuadro de búsqueda de extensiones.

El botón Extensiones:



Cuando pulses sobre dicho botón te aparecerá en la parte superior izquierda del editor un campo que permite buscar extensiones introduciendo su nombre. Una vez encontrada la extensión te la presentará en la parte de arriba de la lista de extensiones. Haz click sobre ella y te aparecerá la información correspondiente a la derecha con un botón **install** para instalarla. Al pulsar sobre install te aparece una ventana como la de abajo:

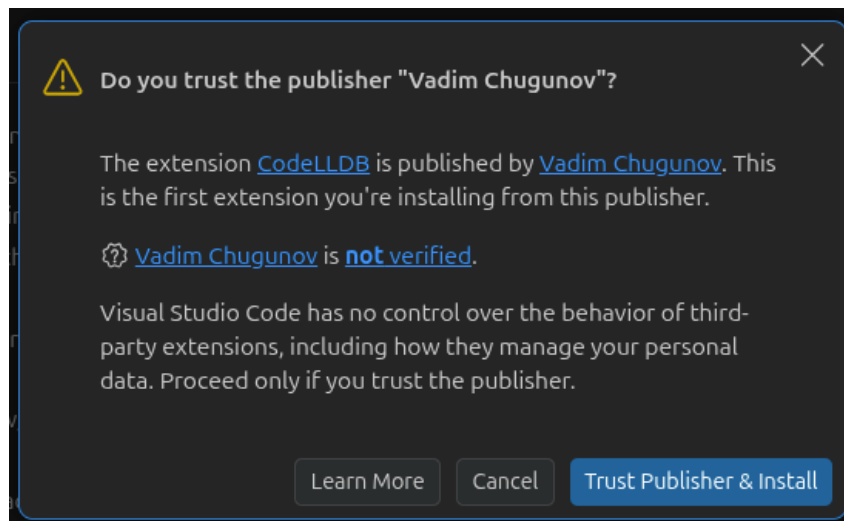


Figura 5: Ventana para aceptar la instalación.

Pulsa sobre **Trust Publisher & Install** para decir que confías en la extensión y se instalará.

Estas son las tres extensiones que tienes que instalar.

1. **Rust Analyzer:** Es el cerebro. Te coloreará el código, te auto-completará palabras y te avisará de los errores en rojo antes de que compiles. (¡Ojo! No instales la que se llama solo Rust, esa es antigua; busca **Rust Analyzer**).
2. **Even Better TOML:** Sirve para que tu ordenador entienda los archivos de configuración de tus proyectos de Rust (los archivos **.toml** donde añadiremos la herramienta **craque** de imágenes).
3. **CodeLLDB:** Es la lupa del detective. Te permitirá pausar tu programa línea a línea si algo falla para ver qué hay dentro de la memoria.

7.5. 🦀 La Gran Comprobación: ¿Funciona todo?

Para asegurarnos de que la instalación ha sido un éxito rotundo, abre una nueva terminal (en Windows puedes usar **Símbolo del sistema** o **PowerShell**; en Linux y Mac, la **Terminal** normal) y escribe estos comandos uno a uno:

```
rustc --version  
cargo --version
```

Si todo ha ido bien, la pantalla no te dará ningún error; te devolverá una línea de texto por cada comando con la versión exacta de Rust que tienes instalada (por ejemplo: rustc 1.85.0 ...).

¡Enhorabuena! Has roto la barrera y ya tienes un entorno de desarrollo profesional en tu ordenador. Estás listo para crear magia con código real y manipular píxeles.

8. 🎨 Tratamiento de imágenes con Rust

Has llegado a la penúltima frontera del primer cuaderno. Hasta ahora, todos tus programas se ejecutaban en la Playground, que es una terminal de texto. En este capítulo vas a dar el salto al mundo visual: vamos a usar Rust para **abrir, leer y modificar** imágenes reales en tu disco duro.

8.1. Creando tu primer proyecto con Cargo

En la Playground solo teníamos el contenido de un archivo volcado en su lado izquierdo. Ni siquiera téamos un fichero físico. En el mundo profesional, un programa real se compone de varios archivos y herramientas. Para gestionarlos, Rust incluye un asistente en la terminal llamado Cargo. **La instalación que hicimos de Rust en el apartado anterior nos proporciona todas las herramientas del asistente Cargo.**

Para organizar nuestros proyectos ya creamos una carpeta raíz de los mismos con el nombre **proyectos-rust**. Puede estar ubicada en cualquier parte de nuestro disco duro. Si has seguido los apartados anteriores de este cuaderno, seguramente ya tendrás creada esta carpeta.

Creación del proyecto

Abre la terminal de tu ordenador en la carpeta **proyectos-rust** y escribe la siguiente orden para crear tu primer proyecto Rust:

```
cargo new tratamiento_imagenes
```

La orden anterior creará la carpeta de proyecto **tratamiento_imagenes** dentro de la primera carpeta creada proyectos-rust.

⚠ El truco de las carpetas:

Cargo es muy estricto y no te permite crear proyectos cuyo nombre empiece por un número (por ejemplo, `8_tratamiento_imagenes` daría error). El truco consiste en crear el proyecto tal y como hicimos arriba y, una vez creado, si lo deseas vas a tu gestor de archivos normal de Windows/Linux/Mac y renombas la carpeta añadiéndole el número delante. ¡Hecha la ley, hecha la trampa!

🔧 Cómo abrir el proyecto en Visual Studio Code:

- Abre Visual Studio Code.
- Ve al menú superior y selecciona **Archivo -> Abrir carpeta** (Open Folder).
- Selecciona la carpeta **proyectos-rust** y dale a **Seleccionar** o **Abrir**.

En la barra lateral izquierda verás la carpeta **proyectos-rust** y en su interior la del proyecto creado anteriormente **tratamiento_imagenes**. Desplegando esta carpeta verás la estructura del proyecto:

- 📁 **src/**: La carpeta sagrada. Dentro está **main.rs**, que es donde escribirás tu código.

-  **Cargo.toml:** El archivo de configuración. Abre este archivo y, debajo de donde dice [dependencies], añade la herramienta de imágenes escribiendo exactamente: **image = "0.24"** (o la versión actual) tal como se muestra a continuación:

```
[dependencies]
image = "0.24"
```

8.2. El cuadro de mandos de Cargo

En primer lugar abre la terminal integrada de VS Code haciendo **click derecho** sobre la carpeta del proyecto y seleccionando **Open in integrated Terminal**. (En español debe ser algo así como Abrir en la terminal integrada)

Para trabajar de forma profesional, solo necesitas memorizar estos comandos que escribirás en la terminal integrada de VS Code cuando corresponda:

Los comandos de Cargo empiezan con **cargo + una palabra más**.

Comando	¿Qué hace?
cargo check	Escanea tu código rápido para ver si tienes errores en rojo sin llegar a compilarlo.
cargo build	Compila tu código y genera el programa ejecutable.
cargo run	Hace todo a la vez: compila el código y lo ejecuta de inmediato en tu pantalla.

8.3. Qué es una imagen para un ordenador?

Para nosotros, una foto es un paisaje o una persona. Para el ordenador, una imagen es una **cuadrícula gigante de colores** (una matriz matemática). Cada cuadradito de esa cuadrícula se llama píxel. Cada rectángulo (debería ser cuadrado) que hemos representado abajo en la matriz representa un píxel de la imagen. Si la imagen fuera por ejemplo de 400x200 píxeles, la matriz que la representa tendrá una anchura de 400 píxeles y una altura de 200 píxeles. Esto equivale a tener 200 filas y 400 columnas.

	Columna 0	Columna 1	Columna 2	...	Columna J
Fila 0	Píxel	Píxel	Píxel		Píxel
Fila 1	Píxel	Píxel	Píxel		Píxel
Fila 2	Píxel	Píxel	Píxel		Píxel
...					
Fila I	Píxel	Píxel	Píxel		Píxel(I, J)

De forma general, al píxel que está situado en la fila I y columna J le llamamos Pixel(I, J)

Cada píxel guarda su color **mezclando tres colores básicos** (Formato RGB: Red, Green, Blue). Por tanto, cada píxel almacena 3 valores numéricos entre 0 y 255. Este rango de valores

entre 0 y 255, ambos inclusive, es el que tienen las variables de tipo u8 (enteros sin signo de 8 bits). En ocasiones, se incorpora un cuarto valor al píxel denominado Alfa que representa la transparencia en imágenes que tienen esa propiedad.

Algunos ejemplos para valores de un píxel:

- **[255, 0, 0]** es Rojo puro (rojo al máximo, cero para verde y cero para azul).
- **[0, 255, 0]** es Verde puro.
- **[0, 0, 255]** es Azul puro.
- **[0, 0, 0]** es Negro (luces apagadas).
- **[255, 255, 255]** es Blanco (luces a tope).

8.4. Programa 1: Leyendo el color de un píxel

Para este experimento, busca una imagen en internet —la famosa foto de prueba **lena.jpg** o cualquier otra; también tienes una carpeta de imágenes en el repositorio que te has descargado —, renómbrala como **entrada.jpg** y guárdala dentro de la carpeta principal de tu proyecto (al lado del fichero **Cargo.toml**).

 **Nota:** Estamos trabajando en el proyecto **tratamiento_imagenes** que creamos en el apartado 8.1.

 Escribe este código dentro de tu archivo **src/main.rs** borrando si existía algo previamente:

```
use image::GenericImageView; // Importamos la herramienta para mirar imágenes

fn main() {
    // 1. Intentamos abrir la imagen de tu disco duro
    let imagen = image::open("entrada.jpg").expect("❌ ¡No encuentro el archivo entrada.jpg!");

    // 2. Le preguntamos sus dimensiones (Ancho y Alto)
    let (ancho, alto) = imagen.dimensions();
    println!("📷 Imagen cargada con éxito. Tamaño: {}x{} píxeles.", ancho, alto);

    // 3. Inspeccionamos las coordenadas de un píxel concreto (Fila 100, Columna 100)
    let pixel = imagen.get_pixel(100, 100);

    // 4. Mostramos sus componentes de color RGB
    println!("🎨 El píxel en (100,100) tiene los valores: RGBA -> {:?}" , pixel);
}
```

Explicación del programa

1. La instrucción:

```
let imagen = image::open("entrada.jpg").expect("❌ ¡No encuentro el archivo entrada.jpg!");
```

Hará que el programa cargue en memoria la imagen **entrada.jpg** si la encuentra en el directorio adecuado (en el directorio principal, al lado de Cargo.toml) y el nombre se corresponde con un fichero de imagen válido. Si no se cumplen estas condiciones, evidentemente no se cargará nada en memoria y se mandará a la terminal el mensaje:

✗ ¡No encuentro el archivo entrada.jpg!

2. La instrucción:

```
let (ancho, alto) = imagen.dimensions();
```

Coloca el ancho en píxeles de la imagen en la variable **ancho** y el alto en píxeles en la variable **alto**.

3. La instrucción:

```
let pixel = imagen.get_pixel(100, 100);
```

Es bastante interesante. La parte que hay a la derecha del igual busca el pixel (100,100) de la imagen y copia sus 4 valores (3 valores de color y uno de transparencia). Los píxeles se cuentan partiendo de la posición (0, 0) en la esquina superior izquierda de la imagen y van aumentando cuando nos desplazamos a la derecha y hacia abajo.

Esos cuatro valores forman un dato compuesto y se asignan en un solo nombre a la variable **pixel**, a la izquierda del igual.

4. La instrucción:

```
println!("👁 El píxel en (100,100) tiene los valores: RGBA -> {:?}", pixel);
```

Muestra los cuatro valores de pixel.

Observa que hemos utilizado el marcador **{:?}** para imprimir **pixel**. Esto es consecuencia de que pixel es una variable compuesta (4 valores) y con este marcador Rust sabe como imprimirla. El resultado de imprimir lo tienes en la línea de abajo que constituye la salida del programa:

```
El píxel en (100,100) tiene los valores: RGBA -> Rgba([139, 71, 60, 255])
```

🚀 Ejecutar el programa

Para compilar el programa necesitas activar una terminal integrada de VS Code apuntando al directorio de tu proyecto. La forma más sencilla de conseguirlo es haciendo click derecho sobre la carpeta del proyecto en el panel izquierdo de VS Code y seleccionar **Open in Integrated Terminal** (Abrir en la Terminal integrada).

Una vez hayas llegado aquí, ejecuta el comando **cargo run** en tu terminal. Verás cómo Rust descompone ese punto exacto de la foto en sus números de color y te entrega el resultado.

8.5. Programa 2: Modificando píxeles y pintando un cuadrado rojo

Vamos a crear un segundo proyecto para modificar píxeles pintando un cuadrado rojo sobre la imagen.

Título del proyecto: **tratamiento_imagenes_2**

Hay varias formas de obtener este segundo proyecto:

1. **La aconsejada:** seguir los pasos indicados en el apartado **8.1 Creando tu primer proyecto con Cargo**
2. Hacer un clon (copiar y pegar) la carpeta de tu primer proyecto **tratamiento_imagenes** y cambiar el título de la carpeta a **tratamiento_imagenes_2**.

Luego, en cualquiera de los dos casos, tendrás que editar el fichero **src/main.rs** y ponerle el código que te daremos más abajo.

En el proyecto del apartado anterior, si hubiéramos procedido a modificar un solo píxel en la foto digital de alta resolución, sería como cambiar un granito de arena en la playa: el ojo humano no se va a dar cuenta. Para comprobar que realmente tenemos el poder de **hackear** la imagen, vamos a hacer un experimento más visible.

Vamos a seleccionar el píxel (100, 100) de la imagen **lena.jpg** (la tienes en la carpeta **imagenes** del repositorio) y, usando la potencia de los bucles que aprendiste en el laboratorio, pintaremos un cuadrado rojo de 11x11 píxeles a su alrededor. Luego, guardaremos el resultado en un archivo nuevo llamado **lena_modificada.png**.

📌 Nota: Es posible que necesites revisar el funcionamiento de los bucles `for` para entender los dos proyectos que siguen a continuación. En tal caso, revisa el apartado '5.4 El bucle `for` con rangos.'

📄 Escribe este código dentro de tu archivo **src/main.rs** borrando primero cualquier código preexistente:

```
use image::{GenericImage, Rgb};

fn main() {
    // 1. Cargamos la imagen original abriendo la caja con permiso de mutación
    (mut)
    // Necesitamos 'mut' porque vamos a alterar sus píxeles.
    let mut imagen = image::open("lena.jpg")
        .expect("¡Error! No se encuentra el archivo lena.jpg");

    println!("🎨 Modificando la imagen... Dibujando zona de pruebas.");

    // El color rojo en formato RGB se compone de:
    // Máximo Rojo (255), cero Verde (0) y cero Azul (0)
    // la variable color_rojo no la utilizamos
    let color_rojo = Rgb([255, 0, 0]);

    // 2. Usamos dos bucles anidados para recorrer un área de 11x11 píxeles
    // El contador 'x' irá desde 95 hasta 105 (11 posiciones en total)
```

```

    for x in 95..=105 {
        // Por cada posición de 'x', el contador 'y' también se mueve de 95 a
105
        for y in 95..=105 {
            // Pintamos el píxel actual con nuestro color rojo
            imagen.put_pixel(x, y, image::Pixel::from_channels(255, 0, 0,
255));
        }
    }

    // 3. Guardamos el resultado en el disco duro con un nombre nuevo
    imagen.save("lena_modificada.png")
        .expect("No se pudo guardar la imagen modificada");

    println!("🖼️ ¡Éxito! Archivo 'lena_modificada.png' guardado en la carpeta
de tu proyecto.");
}

```

⚙️ El análisis del detective: ¿Cómo se pinta un área?

Vamos a destripar la lógica de este programa paso a paso:

- La herramienta **GenericImage**: Al principio del archivo verás que hemos importado `GenericImage`. En el lenguaje de Rust, esta herramienta le da a nuestra imagen el **poder** de ser modificada mediante la función `.put_pixel()`.
- Las variables **mut** de verdad: Fíjate en la línea `let mut imagen = ....` Si hubiéramos olvidado poner la palabra `mut`, Rust habría bloqueado el programa en seco al llegar a la orden de pintar. Rust protege la imagen original a menos que le digas explícitamente con **mut** que la vas a alterar.
- El escáner del bucle (`for x in 95..=105`): La sintaxis `95..=105` en Rust significa recorre todos los números desde el 95 hasta el 105, ambos incluidos. Al meter un bucle dentro de otro, el ordenador funciona como una máquina de coser: se posiciona en la **columna x = 95** y recorre **todas las filas de esa columna** pintando de rojo, luego pasa a **x = 96** y repite. Así hasta rellenar el cuadrado perfecto de 11x11.

🎮 ¡Tu momento de comprobar el laboratorio!

1. Abre tu terminal integrada en VS Code sobre el directorio del proyecto y escribe tu comando sagrado:

```
cargo run
```

2. Al terminar, fíjate en la barra lateral izquierda de VS Code (donde están los archivos de tu proyecto). ¡Magia! Ha aparecido un archivo nuevo llamado **lena_modificada.png**.
3. Haz clic sobre él dentro de VS Code para abrirlo. Busca con la mirada la zona superior izquierda, coordenadas (100, 100). Verás un pequeño y perfecto cuadrado de color rojo brillante flotando sobre la imagen. ¡Has alterado con éxito los píxeles usando tu propio código!

🎨 El resultado que debes obtener



Figura 6: Imagen de Lena con el cuadrado rojo superpuesto

8.6. Programa 3: Recorriendo la matriz completa (El filtro Negativo)

Ya sabemos localizar un píxel y sabemos pintar un cuadrado. Ahora vamos a hacer magia a lo grande: vamos a aplicar un filtro de colores invertidos (efecto negativo) a toda la fotografía.

Nota: Crearemos un nuevo proyecto llamado 'tratamiento_imagenes_3' siguiendo los mismos pasos que en el apartado anterior.

Para lograrlo, le diremos a Rust que averigüe el **ancho** y el **alto** exactos de la imagen y que pase un escáner píxel por píxel, desde la esquina superior izquierda hasta la esquina inferior derecha.

La regla matemática para invertir un color es muy sencilla: **restamos el valor actual de cada canal (Rojo, Verde y Azul) al número máximo posible, que es 255.**

🧙 Tu primer superpoder de edición: ¡El Filtro Negativo! ¿Sabías que los filtros de Instagram o de TikTok son en realidad pequeños programas matemáticos que modifican los píxeles uno a uno? En este experimento vamos a crear nuestro propio filtro.

Un ordenador ve los colores combinando Rojo (Red), Verde (Green) y Azul (Blue) con valores de 0 a 255. Para hacer el negativo, solo tenemos que restarle a 255 el valor actual de cada color.

Si un píxel es muy rojo (255), tras la resta pasará a ser 0 (nada de rojo). ¡Mira el resultado (al final del apartado) de aplicar este bucle matemático a nuestra imagen de prueba! ¿A que parece sacada de una película de ciencia ficción?

 **Escribe este código dentro de tu archivo src/main.rs:**

```
use image::{GenericImage, GenericImageView, Pixel};

fn main() {
    // 1. Cargamos la imagen original con permiso para modificarla (mut)
    let mut imagen = image::open("lena.jpg")
        .expect("¡Error! No se encuentra el archivo lena.jpg");

    // 2. Le preguntamos a Rust cuáles son las dimensiones de la foto
    let (ancho, alto) = imagen.dimensions();
    println!("🖼️ Imagen cargada correctamente (Tamaño: {} x {} píxeles).",
ancho, alto);
    println!("⌚ Aplicando filtro negativo a toda la imagen... Esto puede
tardar un par de segundos.");

    // 3. El gran escáner: recorremos todas las columnas (x) y todas las filas
    (y)
    for x in 0..ancho {
        for y in 0..alto {
            // Capturamos el píxel actual en esa coordenada
            let pixel_original = imagen.get_pixel(x, y);

            // Extraemos sus canales de color en formato de lista (Rojo, Verde,
Azul, Alfa)
            let canales = pixel_original.to_rgba();

            // Calculamos el color invertido restando cada componente a 255
            let nuevo_rojo = 255 - canales[0];
            let nuevo_verde = 255 - canales[1];
            let nuevo_azul = 255 - canales[2];
            let alfa = canales[3]; // La transparencia la dejamos exactamente
igual

            // Fabricamos el nuevo píxel con los colores calculados
            let nuevo_pixel = Pixel::from_channels(nuevo_rojo, nuevo_verde,
nuevo_azul, alfa);

            // Inyectamos el nuevo píxel de vuelta en la imagen tapando el
viejo
            imagen.put_pixel(x, y, nuevo_pixel);
        }
    }

    // 4. Guardamos la nueva obra de arte
    imagen.save("lena_negativo.png")
        .expect("No se pudo guardar la imagen modificada");

    println!("📁 ¡Filtro completado! Revisa el archivo 'lena_negativo.png'.");
}
```

⚙️ El análisis del detective: ¿Cómo escaneamos miles o millones de píxeles?

- El radar automático **imagen.dimensions()**: Gracias a la herramienta **GenericImageView**, no necesitamos adivinar el tamaño de la foto. Rust lee el archivo y nos devuelve dos números enteros: el **ancho** y el **alto**.
- La lista de canales (**canales[0]**, **canales[1]**, **canales[2]**, **canales[3]**): Un píxel guarda sus cuatro valores como si fuera una lista de elementos en un estante. En programación, siempre empezamos a contar desde el cero:

```
canales[0] es el Rojo
canales[1] es el Verde
canales[2] es el Azul
canales[3] es la Transparencia
```

- La ilusión de la velocidad: Aunque parezca que el ordenador hace un trabajo titánico recorriendo miles de píxeles uno a uno, gracias al espectacular rendimiento de Rust, el bucle procesará toda la matriz en menos de lo que tardas en parpadear.

🎮 ¡Comprueba tu filtro profesional!

1. Crea una terminal integrada de VS Code sobre la carpeta del proyecto.
2. Ejecuta tu comando en la terminal:

```
cargo run
```

3. Abre el nuevo archivo generado **lena_negativo.png**. ¡Verás la mítica foto transformada por completo en una película de terror o un negativo fotográfico antiguo! Los rostros claros ahora son oscuros, y los colores han cambiado por sus opuestos exactos.

🎨 El resultado que debes obtener



Figura 7: Imagen de Lena con un filtro negativo

9. Conceptos avanzados en Rust

Hemos querido llegar hasta el final del Cuaderno 1 para introducir algunos conceptos que son fundamentales en Rust y que a su vez lo diferencian de otros lenguajes de programación.

Rust basa la seguridad en la memoria en el concepto de **ownership / borrowing**, que podemos traducir como **propiedad / préstamo o referencia**.

El concepto de propiedad es una especie de simil que se aplica a variables que son propietarias del dato que contienen y el de préstamo o referencia a variables que toman prestado el dato. Como veremos a continuación, las consecuencias que se derivan en ambos casos son diferentes.

Las variables propietarias son las encargadas de liberar la memoria que ocupa su dato cuando éste ya no se necesita.

Primera identificación visual

En los programas veremos frecuentemente representados estos tipos de variables como se muestra a continuación. Para crear una referencia se utiliza el operador `&`:

 Ejecuta el siguiente programa en la Playground.

```
fn main() {
    // variable propietaria (variable normal)
    let nombre = String::from("Antonio");

    // préstamo o referencia a la variable 'nombre'
    let ref_nombre = &nombre;

    // Podemos acceder a la variable 'nombre' a través de una referencia
    println!("{}", ref_nombre);    // Antonio

    // <--- Aquí la variable 'nombre' sigue activa
} // <--- Al llegar a la llave de cierre, 'nombre' es destruida por Rust
```

Cuando la ejecución del programa llega a la **llave de cierre del main**, la variable **nombre** es destruida por Rust y se **libera la memoria** que tenía reservada para almacenar el string. Decimos entonces que **nombre** sale de su contexto. El **contexto de nombre** abarca desde su definición hasta la siguiente llave de cierre.

Esta liberación de memoria es fundamental para que las posteriores operaciones sobre la memoria sean seguras.

Las **referencias o préstamos** mantienen siempre esa **dualidad conceptual**.

El **Concepto de préstamo** se fundamenta en que a través de **ref_nombre** en el programa anterior, podemos acceder a la variable original **nombre** pero la liberación de la memoria es siempre obra de la variable original. Por eso decimos que la referencia **ref_nombre** tiene un préstamo del dato.

El **Concepto de Referencia** (Dirección de Memoria y Desreferenciación)

Para explicar qué es una referencia, podemos combinar la idea del «préstamo» con la realidad física de la memoria.

Una **referencia** (&) suele definirse intuitivamente como un «préstamo» de datos, pero a nivel físico es algo más preciso: representa la **dirección inicial de memoria** donde reside una variable.

Al crear una referencia, no duplicamos el contenido del dato, sino que apuntamos a su ubicación original. Para realizar el camino inverso y acceder directamente al valor contenido en esa dirección de memoria, utilizamos el operador de **desreferenciación** (*).

Idea clave: Una referencia es una dirección de memoria que apunta a un dato. El operador * sigue esa dirección para leer o modificar el valor real.

 Ejecuta el siguiente programa en la Playground.

```
fn main() {
    // original es propietaria del dato 42
    let original = 42;

    // Guarda la dirección de memoria de 'original'
    let referencia = &original;

    // Para acceder al valor apuntado, usamos el operador de
    // desreferenciación (*)
    let valor_copiado = *referencia;

    println!("Dirección apuntada (referencia): {:p}", referencia);
    println!("Valor desreferenciado: {}", valor_copiado);    // 42

    println!("Valor de referencia: {}", referencia);          // 42
}
```

- Decimos que la variable **referencia** toma prestado el dato 42.
- Utilizamos el marcador **{:p}** para imprimir una dirección de memoria.
- Obtenemos el valor de la variable **original** mediante el operador *:

```
let valor_copiado = *referencia;
```

Sie embargo, la siguiente instrucción imprime el valor de la variable original sin utilizar el operador *:

```
println!("Valor de referencia: {}", referencia);    // 42
```

Esto es consecuencia de que la macro **println!()** interpreta por defecto que cuando le pasamos una dirección, lo que queremos en realidad es que imprima el contenido de esa dirección. Para imprimir la dirección tenemos que utilizar el marcador **{:p}** como hemos hecho antes.

9.1. Ámbito (Scope) y la Destrucción de Datos (drop)

A diferencia de otros lenguajes con recolector de basura (Garbage Collector) o gestión manual, Rust utiliza un sistema basado en el **ámbito** (las llaves `{ }`).

La regla de oro del Ownership es simple: una variable es la dueña exclusiva de su dato. En el instante exacto en que esa variable sale de su ámbito (es decir, el flujo del programa llega al cierre de la llave `}`), Rust invoca automáticamente una función especial llamada `drop`. Esta función devuelve inmediatamente la memoria asignada al sistema operativo, garantizando la seguridad sin penalizaciones en tiempo de ejecución.

📌 **Nota:** El siguiente programa utiliza un tipo de dato **struct Recurso** que todavía no hemos estudiado. Si quieres saber cómo funciona está explicado en el apartado «9.7 El struct Persona y los superpoderes automáticos»

 Copia este código y ejecútalo en tu Playground

```
struct Recurso {
    nombre: String,
}

// Implementamos Drop para "ver" cuándo se destruye
impl Drop for Recurso {
    fn drop(&mut self) {
        println!("¡Liberando memoria de {}!", self.nombre);
    }
}

fn main() {
    println!("Inicio del main");
    {
        let _mi_recurso = Recurso { nombre: String::from("Cuaderno 1") };
        // _mi_recurso está vivo aquí
    } // <-- Aquí se cierra el ámbito. Rust llama automáticamente a drop()

    println!("Fin del main (el recurso ya no existe)");
}
```

Salida del programa:

```
Inicio del main
¡Liberando memoria de Cuaderno 1!
Fin del main (el recurso ya no existe)
```

Para entender este programa vamos a tener que hacer un pequeño acto de fe.

- Con el bloque **impl**:

```
impl Drop for Recurso {
    fn drop(&mut self) {
        println!("¡Liberando memoria de {}!", self.nombre);
    }
}
```

```
}
}
```

Definimos el método **drop** (similar a una función) para el **struct Recurso**. Este método **drop** es el que llama Rust cuando va a destruir una variable de tipo Recurso, aparte por supuesto de destruir la variable liberando la memoria asociada.

Entonces, cuando se destruya la variable `_mi_recurso` se imprimirá también el mensaje:

```
¡Liberando memoria de Cuaderno 1!
```

❗ **Nota:** el guión bajo inicial en la variable `_mi_recurso` es simplemente para que el compilador no te muestre el mensaje de que no utilizas la variable después de su declaración. Si lo quitas funcionará igual pero tendrás el mensaje de aviso del compilador.

- Empezando por la primera instrucción de la función `main`, sigue el flujo del programa para obtener los mensajes de la salida del programa.

9.2. Conflicto: Préstamo Mutable e Intento de Lectura Posterior

Rust prohíbe en tiempo de compilación usar la variable original mientras exista un préstamo mutable activo, evitando carreras de datos: cuando dos variables intentan acceder simultáneamente al mismo dato (por supuesto, en tiempo de ejecución).

Préstamo mutable

Al principio de este capítulo hemos definido el concepto de referencia o préstamo y como se obtiene a partir de la variable original. A partir de una referencia podemos acceder al valor de la variable original como se muestra a continuación.

```
fn main() {
    // original es propietaria del dato 42
    let original = 42;

    // Guarda la dirección de memoria de 'original'
    let referencia: &i32 = &original;

    // Para acceder al valor apuntado, usamos el operador de
    // desreferenciación (*)
    let valor_copiado = *referencia;

    println!("Valor desreferenciado: {}", valor_copiado);
}
```

A partir de `*referencia` podemos leer el valor original. Sin embargo no podemos modificarlo. Para poder modificarlo tenemos que obtener una referencia mutable `&mut` como en el siguiente programa:

```
fn main() {
    let mut original = 42;

    // Referencia mutable
    let referencia = &mut original;

    *referencia +=100;

    println!("Valor original: {}", original); // Valor original: 142
}
```

- Como vamos a modificarla, la variable **original** debe ser **mutable (mut)**.
- Como podemos comprobar, a través de la referencia mutable hemos cambiado el valor de la variable original de 42 a 142.

Regla del compilador

El compilador de Rust (mediante el **Borrow Checker**) aplica una regla matemática estricta para evitar carreras de datos en memoria:

- Puedes tener tantos préstamos inmutables (**&T**) como quieras al mismo tiempo.
- **O bien** puedes tener **un único** préstamo mutable (**&mut T**) a la vez.

 **Nota:** T representa un tipo de dato cualquiera

Si intentas leer o usar la variable original mientras un préstamo mutable esté activo y en uso, el compilador detendrá la ejecución inmediatamente. La variable original queda «bloqueada» hasta que el préstamo mutable termine por completo su ciclo de vida.

 Copia este código y ejecútalo en tu Playground

```
fn main() {
    let mut texto = String::from("Hola");

    let prestamo_mutable = &mut texto; // Se crea el préstamo mutable
    prestamo_mutable.push_str(" Rust");

    // ERROR DE COMPILACIÓN: No podemos usar 'texto' aquí porque
    // 'prestamo_mutable' todavía está activo en la línea siguiente
    // ⬇️ ❌ --- error, no compilará
    // println!("{}", texto); // <--- DESCOMENTAR

    // Si usamos el préstamo mutable primero:
    println!("Modificado: {}", prestamo_mutable);

    // Aquí el préstamo mutable termina su ciclo de vida.
    // Ya no se utiliza más hasta el final del programa.
    // Ahora sí podemos volver a usar la variable original de forma segura:
    println!("Original liberada: {}", texto);
}
```

Cuando el compilador va a compilar una línea, no mira solo las líneas anteriores para ver si puede compilar sino que mira también las líneas hasta el final de la siguiente llave.

Si descomentamos la línea: `println!("{}", texto);`, el compilador ve que hay una línea posterior: `println!("Modificado: {}", prestamo_mutable);` en la que estamos utilizando `prestamo_mutable` y eso está prohibido. No compilará. Es la regla del compilador que hemos visto antes: no podemos utilizar la variable propietaria `texto` si después utilizamos `prestamo_mutable` por ejemplo para imprimirlo.

Cuando en la última línea utilizamos la variable `texto` ya no hay problema porque después ya no utilizamos `prestamo_mutable`.

9.3. Pasar Variables a Funciones (&, &mut y Move)

Cuando enviamos una variable como argumento a una función, el comportamiento varía drásticamente según la firma del método:

 **Nota:** T representa un tipo de dato cualquiera

Supongamos que desde la función `main` llamamos a otra función de un solo parámetro según los casos que se discuten a continuación. Como argumento para ese parámetro le pasaremos una variable de nuestra función `main`:

- **Por referencia inmutable:**

- Tipo del parámetro: `(var: &T)`

Prestamos la variable solo para lectura. La función no puede modificarla y nosotros conservamos la propiedad al terminar la función.

- **Por referencia mutable:**

- Tipo del parámetro: `(var: &mut T)`

Prestamos la variable con permisos de escritura. La función puede alterarla directamente en memoria y nosotros conservamos el control de la variable modificada al terminar la función.

- **Por valor:**

- Tipo del parámetro: `(var: T)`

Ocurre un **Move** (movimiento de la propiedad). Transferimos por completo la propiedad de la variable a la función. Al acabar la función, la variable original se destruye (`drop`) y no podremos volver a usarla en nuestro código principal.

Este triple ejemplo es perfecto para contrastar las diferencias en las tres formas de pasar los argumentos:

 Copia este código y ejecútalo en tu Playground

Fichero: `pasar_parametro_funcion.rs`

```
fn solo_lectura(texto: &String) {
    println!("Leemos sin alterar: {}", texto);
}
```

```
fn modificacion(texto: &mut String) {
    texto.push_str(" modificado");
}

fn toma_propiedad(texto: String) {
    println!("Tengo la propiedad de: {}", texto);
} // <-- Aquí se hace drop de 'texto'

fn main() {
    let mut mi_cadena = String::from("Datos");

    // Préstamo inmutable (conservo propiedad de mi_cadena)
    solo_lectura(&mi_cadena);

    // Préstamo mutable (conservo propiedad y modifico mi_cadena)
    modificacion(&mut mi_cadena);

    // Move o Transferencia de propiedad de mi_cadena a la función
    // Después de esta instrucción mi_cadena ya no existe
    toma_propiedad(mi_cadena);

    // ⬇️ ❌ --- error: mi_cadena fue destruida en la función.
    // println!("{}", mi_cadena);
}
```

9.4. El Comportamiento en los Bucles for (Iteradores)

Con el bucle for iteramos entre los distintos elementos de una colección: un array, un vector, etc.

Dependiendo de cómo le pasemos la colección, el bucle alterará la propiedad o los permisos de los elementos:

A) Préstamo Inmutable

- `for elemento in &coleccion`
 - Extrae en cada elemento una referencia de lectura (`&T`). La colección original sigue intacta y usable tras el bucle for.
 - Equivale a llamar a `.iter()`.

B) Préstamo Mutable

- `for elemento in &mut coleccion`
 - Extrae en cada elemento una referencia mutable (`&mut T`). Permite modificar los elementos dentro del bucle. La colección se conserva después del bucle.
 - Equivale a llamar a `.iter_mut()`.

C) Consumo / Move

- `for elemento in coleccion`
 - Consume los elementos por valor (`T`), transfiriendo la propiedad al bucle. No podemos modificar los elementos. Al terminar, la colección original queda destruida y ya no existe; ya no puedes usarla.
 - quivale a `.into_iter()`.

D) Consumo / Move (con posibilidad de modificar los elementos)• **for mut elemento in coleccion**

- Consume los elementos por valor (τ), transfiriendo la propiedad al bucle. Podemos modificar los elementos. Al terminar, la colección original queda destruida y ya no existe; ya no puedes usarla.
- quivale también a `.into_iter()`.

 Copia este código y ejecútalo en tu Playground

Fichero: **iteradores_for.rs**

```
fn main() {
    // Definimos un vector de tres elementos
    let mut numeros = vec![10, 20, 30];

    // A) Préstamo inmutable
    for num in &numeros {          // num es &i32
        // num es una referencia pero println!() imprime
        // el valor almacenado en la referencia.
        println!("Varlor referenciado: {}", num);

        // También podemos imprimir el valor
        // almacenado en la referencia con:
        println!("Valor: {}", *num);
    }

    // B) Préstamo mutable
    for num in &mut numeros {      // num es &mut i32
        *num += 1; // Modificamos el valor desreferenciando
        println!("Elemento modificado: {}", num);
    }

    // C) Move (Consumo total)
    for num in numeros {           // num es i32
        println!("Elemento propio: {}", num);
    }

    //   -- ERROR: numeros ya no existe
    // println!("{}", numeros);
}
```

9.5. Iteradores que devuelven referencias (Ejemplo Extra)

A veces usamos métodos como `.iter()` o `.find()` fuera de un bucle for. Es útil mostrar cómo estos métodos devuelven referencias y cómo tratarlas.

 Copia este código y ejecútalo en tu Playground

Fichero: **iter.rs**

```
fn main() {
    let nombres = vec![String::from("Halcón"), String::from("Rust")];

    // .iter() devuelve referencias (&String)
```

```

let mut iterador = nombres.iter();

if let Some(nombre_ref) = iterador.next() {
    // nombre_ref es un &String. No podemos moverlo, solo leerlo.
    println!("Encontrado: {}", nombre_ref);    // Halcón
}

if let Some(nombre_ref) = iterador.next() {
    // nombre_ref es un &String. No podemos moverlo, solo leerlo.
    println!("Encontrado: {}", nombre_ref);    // Rust
}

// La lista 'nombres' sigue perfectamente accesible aquí.
println!("Lista completa: {:?}", nombres);
}

```

Este programa utiliza **Some** que todavía no hemos estudiado.

Para entender la parte del iterador no es imprescindible entender **Some**.

Hemos construido un **iterador** a partir del vector **nombres** mediante la instrucción:

```
let mut iterador = nombres.iter();
```

Mediante las siguientes instrucciones:

```

if let Some(nombre_ref) = iterador.next() {
    // nombre_ref es un &String. No podemos moverlo, solo leerlo.
    println!("Encontrado: {}", nombre_ref);
}

```

iterador.next() proporciona una referencia **&String** al elemento correspondiente del vector **nombres**. La primera vez al primer elemento "Halcón" y la segunda vez que las utilizamos al segundo elemento "Rust"

Esa referencia se coloca en la variable **nombre_ref** que va cambiando su contenido y la imprimimos.

9.6. El struct Persona y los «superpoderes» automáticos

A lo largo de este viaje has aprendido que en Rust existen cajas fijas para guardar números enteros (i32), números decimales (f64) o cadenas de texto (&str). Pero, ¿qué pasa si queremos crear nuestra propia **caja personalizada**?

Imagina que estamos programando una utilidad para el instituto o un videojuego de rol, y necesitamos guardar los datos de los usuarios. En lugar de tener tres variables sueltas por el código para el **nombre**, la **edad** y la **profesion**, Rust nos permite inventar nuestro propio tipo de dato usando la palabra mágica **struct** (abreviatura de estructura).

 **Escribe este código dentro de tu archivo src/main.rs:**

Limpia tu archivo **main.rs** en VS Code y escribe este último programa de nivel avanzado:

```
// 💡 la etiqueta #[derive(Debug)] le da al struct el "superpoder"
// de poder imprimirse en pantalla (es un dato compuesto)

// Definimos la estructura
#[derive(Debug)]
struct Persona {
    nombre: String,
    edad: i32,
    profesion: String,
}

fn main() {
    println!("🇪🇸 CREANDO FICHA DE PERSONA EN LA MEMORIA 🇪🇸\n");

    // 1. Rellenamos la ficha creando un objeto con nuestra estructura
    // personalizada, la que hemos creado arriba
    let usuario = Persona {
        nombre: String::from("Halcón68"),
        edad: 14,
        profesion: String::from("Programador de Rust"),
    };

    // 2. Método 1: Leer e imprimir los campos uno por uno
    // (usando el punto '.')
    println!("👤 Nombre del usuario: {}", usuario.nombre);
    println!("🎂 Edad actual: {} años", usuario.edad);
    println!("💼 Profesión: {}", usuario.profesion);

    println!("\n-----\n");

    // 3. Método 2: Imprimir la estructura COMPLETA de golpe
    // ⚠️ ¡Ojo! Para imprimir un struct entero usamos el marcador
    // especial {:?} y tenemos que haberle dado superpoderes antes con:
    // #[derive(Debug)]
    println!("📷 Radiografía completa del objeto en memoria:\n {:?}", usuario);
}
```

⚙️ El análisis del detective: Perdiendo el miedo a internet

Si buscas códigos de Rust en foros o tutoriales de internet, te vas a cruzar constantemente con líneas raras que llevan un signo de almohadilla y corchetes, como `#[derive(Debug)]`. Vamos a quitarles la máscara para que veas que no muerden:

- **¿Qué es un struct?:** Piensa en él como el diseño en papel de una ficha de estudiante. No es un dato real todavía, es solo la plantilla que dice: «Cualquier Persona que creemos con esta estructura tendrá obligatoriamente un **nombre**, una **edad** y una **profesión**». Estas tres variables se denominan campos y cuando diseñamos el **struct** especificamos de que tipo deben ser.

A partir del **struct Persona** hemos creado una variable **usuario** que será de tipo **Persona**, pasando valores a los campos del struct con unos tipos que coinciden con los especificados en el struct Persona.

- **El operador punto (usuario.nombre):** Para acceder a los campos guardados dentro de nuestra estructura, usamos un punto. Es la forma de decirle a Rust: ve a la caja llamada usuario y, por ejemplo, sácame únicamente lo que haya en su cajón nombre.
- **Las directivas o «Superpoderes»** `#[derive(Debug)]`: Por defecto, Rust es tan estricto con la eficiencia que no sabe cómo imprimir una estructura completa en pantalla con un `println!()` normal. Si intentas poner solo el marcador `{}`, el compilador te dará un error rojo gigante. Al escribir `#[derive(Debug)]` justo encima del **struct**, le estamos inyectando un superpoder automático para que Rust aprenda a hacerle una radiografía visual a toda la estructura cuando usemos el marcador especial `{:?}` y así pueda imprimirlo.

¡Haz la prueba técnica!

Abre una terminal integrada para tu proyecto y ejecútalo con el comando que ya te sabes de memoria:

```
cargo run
```

Verás cómo la terminal imprime primero los datos limpios uno por uno, y al final te muestra la radiografía exacta del objeto tal y como vive dentro de la memoria de tu ordenador:

```
Persona { nombre: "Halcón68", edad: 14, profesion: "Programador de Rust" }.
```

9.7. ¿Qué son los traits?

Un Trait (que significa **rasgo o característica** en inglés) es simplemente una lista de tareas o habilidades que un tipo de datos promete saber hacer. En otros lenguajes de programación se le conoce como una Interfaz.

Decimos que un tipo de datos implementa un trait cuando posee todas las habilidades que componen el trait. El trait se implementa mediante código.

Veamos un ejemplo muy sencillo. Definimos el trait **Hablador** con un solo método, **hacer_sonido(&self)**. Los traits pueden contener cualquier número de métodos. Cuando se definen los métodos en el trait solo se pone su cabecera. Los tipos que implementen el trait tienen que dar código a esos metodos cuya cabecera heredan del trait.

En el ejemplo, los **struct Perro** y **Pato** implementan (con la instrucción **impl**) el trait **Hablador** y dan código cada uno por su cuenta al método **hacer_sonido()**. Podemos ver en el main como se instancian los objetos **perro** y **pato** y llaman a su propio método **hacer_sonido()**.

 Copia el siguiente listado en la Playground y ejecútalo.

Fichero: **traits.rs**

```
/*
--- Este es el contrato ---
Cualquier tipo (de dato) que sea un "Hablador"
debe saber hacer_sonido
*/
```

```
// Definimos el trait Hablador
trait Hablador {
    fn hacer_sonido(&self);
}

// Estructura del objeto perro
struct Perro {
    nombre: String,
}

// Estructura del objeto pato
struct Pato {
    nombre: String,
}

// El Perro firma el contrato "Hablador"
impl Hablador for Perro {
    fn hacer_sonido(&self) {
        println!("¡Guau! Soy el perro {}", self.nombre);
    }
}

// El Pato firma el contrato "Hablador"
impl Hablador for Pato {
    fn hacer_sonido(&self) {
        println!("¡Cuaic! Soy el pato {}", self.nombre);
    }
}

fn main() {
    let perro = Perro {
        nombre: String::from("Jerry"),
    };

    let pato = Pato {
        nombre: String::from("Lucas"),
    };

    perro.hacer_sonido();
    pato.hacer_sonido();
}
```

self significa el propio objeto que ejecuta el método.

Por ejemplo, en la instrucción:

- `perro.hacer_sonido();`

Como el método `.hacer_sonido(&self);` recibe una referencia (**&self**) al objeto **perro**, puede acceder a su nombre mediante **self.nombre**.